

期中复习

week1

- 创始人：Guido 龟叔

python语言编程基础

- 用缩进表示代码块
 - 缩进可以使用任意数量的空格或制表符，只要同级代码块保持一致即可
- 关键字和大小写
 - Python语句对大小写敏感
关键字的大小写不能写错
大小写不同，变量无关系
- 代码注释
 - 多行注释是三个英文的单引号或双引号作为注释的开始与结束
 - 单行注释用#表示注释开始，#注释后面内容不会被执行
单行注释可以占一行
- 语句续行
 - Python一句语句占一行
 - 如果很长的语句，可以使用续行符“\”，注意后面不可以有任何其他符号，包括空格和注释
 - 如果在使用括号（）[]{}时，括号中的内容可以直接分多行书写。括号中的空白和换行符会被忽略
- 变量命名
 - 变量名不能以数字开头，不能包含连字符和空格，下划线允许
- 基本输入input
变量=input（“提示字符串”）
 - Input函数将用户输入以字符串返回，用户按【enter】键完成输入【enter】键前所有字符均作为输入内容
 - type(a) 读取变量a的类型
- 基本输出print
print(value, ..., sep=' ', end='\n', file=sys.stdout, flush=False)
 - value:输出的内容，数值直接写，字符串加单引号或双引号
 - sep: 在值之间插入的字符串，默认为空格。

end: 在最后一个值后附加的字符串，默认为换行符。
file: 类似于文件的对象（流）；默认为当前系统标准输出（通常为屏幕）
Flush: 强制刷新，默认False，不刷新，True时刷新

```
print('python')
print('is good')
输出: python
      is good
print('python', end=' ')
print('is good')
输出: python is good
why?: print()默认结尾加\n, print(end=' ')手动修改end参数, 打印完内容之后, 结果不换行, 结尾加空格
```

 print()可以输出多个值，用逗号分隔

- range
 - range (start,end,step): 创建一个整数列表，从start开始，end结束（不包括end）。
range (10) :rang (0, 10, 1), 10个
- eval()
 - 把字符串当成有效的表达式来求值并返回计算结果
 - 计算数学表达式
result = eval("1 + 2")
print(result) # 输出 3
 - 把字符串转化为python对象
eval("[1, 2, 3]") # 得到列表 [1, 2, 3]
eval("{'a':1, 'b':2}") # 得到字典 {'a':1, 'b':2}
- 基本数据类型
 - 整形数 int
 - 不带有小数部分的数字，包括自然数，零，负自然数
十进制表示方式如：10
二进制表示方式（0b开头，后跟0~1） 如：0b111
八进制表示方式（0o开头，后跟0~7） 如：0o177
十六进制表示方式（0x开头，后跟0~9，A~F,a~f） 如： 0xFF
 - 浮点数float

- 浮点型数是带有小数部分的数字（小数部分也可以是0）
小数 如： 3.14 指数形式，小数e/E(正负号) 指数 如： 6.18E-1
- 复数 complex
 - 分为两部分：实部（real）虚部（image）虚部以j或J结尾
字面值表示方式，如： 1+2j
- 布尔型数boo
 - 用来表示真假两种状态的类型
True 表示真（条件满足或成立），1, False 表示假（条件不满足或不成立），0
- 空值对象 None
 - None是一个表示不存在的特殊对象，用于占位或变量解除绑定

💡 Python没有独立的char类型，字符用长度为1的字符串表示

- 基本运算符
 - 算术运算符

运算符	描述	实例
+	加 - 两个对象相加	a + b 输出结果 30
-	减 - 得到负数或是一个数减去另一个数	a - b 输出结果 -10
*	乘 - 两个数相乘或是返回一个被重复若干次的字符串	a * b 输出结果 200
/	除 - x除以y	b / a 输出结果 2
%	取模 - 返回除法的余数	b % a 输出结果 0
**	幂 - 返回x的y次幂	a**b 为10的20次方， 输出结果 100000000000000000000
//	取整除 - 返回商的整数部分	9//2 输出结果 4 , 9.0//2.0 输出结果 4.0

Python 中的整数除法

在 Python 中，除法运算有多种方式，适用于不同场景。以下是常见的除法运算符及其用法：

1. 除法运算符

运算符	名称	说明	示例	↓
/	真除法	返回浮点数结果	7 / 2	3.5
//	地板除法	返回向下取整的整数商	7 // 2	3
%	取模运算	返回除法的余数	7 % 2	1
divmod()	内建函数	同时返回商和余数的元组	divmod(7, 2)	(3, 1)

💡 // 地板除 除的结果去掉小数部分向下取整（比实际值小的整数），且输出为整数不带点

在遇到不同类型的数字参数运算时，Python总是将简单类型转换为复杂类型

- 位运算符
 - &：按位与运算符
 - 参与运算的两个值，如果两个相应位都为1，则该位的结果为1，否则为0；
 - |：按位或运算符
 - 只要对应的二个二进位有一个为1时，结果位就为1；
 - ^：按位异或运算符
 - 当两对应的二进位相异时，结果为1；
 - ~：按位取反运算符
 - 对数据的每个二进制位取反，即把1变为0，把0变为1；
 - >>：右移动运算符
 - 运算数的各二进位全部右移若干位，高位补0，低位丢弃
 - <<：左移动运算符
 - 运算数的各二进位全部左移若干位，由 << 右边的数指定移动的位数，高位丢弃，低位补0

💡 十进制转二进制： bin (a) binary表示二进制
二进制转十进制： a=int("000010101",2)

💡 a = 0001 0101
b = 0000 0110
a&b = 0000 0100
a|b = 0001 0111
a^b = 0001 0011
~a = 1110 1010
a<<2 = 0101 0100
a>>2 = 0000 0101

- 赋值运算符

运算符	描述	实例
=	简单的赋值运算符	c = a + b 将 a + b 的运算结果赋值为 c
+=	加法赋值运算符	c += a 等效于 c = c + a
-=	减法赋值运算符	c -= a 等效于 c = c - a
*=	乘法赋值运算符	c *= a 等效于 c = c * a
/=	除法赋值运算符	c /= a 等效于 c = c / a
%=	取模赋值运算符	c %= a 等效于 c = c % a
**=	幂赋值运算符	c **= a 等效于 c = c ** a
//=	取整除赋值运算符	c //= a 等效于 c = c // a

• 比较运算符

比较运算符	功能
>	大于，如果运算符前面的值大于后面的值，则返回 True；否则返回 False
>=	大于或等于，如果运算符前面的值大于或等于后面的值，则返回 True；否则返回 False
<	小于，如果运算符前面的值小于后面的值，则返回 True；否则返回 False
<=	小于或等于，如果运算符前面的值小于或等于后面的值，则返回 True；否则返回 False
==	等于，如果运算符前面的值等于后面的值，则返回 True；否则返回 False
!=	不等于，如果运算符前面的值不等于后面的值，则返回 True；否则返回 False
is	判断两个变量所引用的对象是否相同，如果相同则返回 True
is not	判断两个变量所引用的对象是否不相同，如果不相同则返回 True

• 逻辑运算

运算符	描述	实例
and	布尔"与" - 如果x为False，x and y返回False，否则它返回y的计算值。	(a and b) 返回 true。
or	布尔"或" - 如果x是True，它返回True，否则它返回y的计算值。	(a or b) 返回 true。
not	布尔"非" - 如果x为True，返回False。如果x为False，它返回True。	not(a and b) 返回 false。

• 成员运算

运算符	描述	实例
in	如果在指定的序列中找到值返回True，否则返回False。	x 在 y序列中，如果x在y序列中返回True。
not in	如果在指定的序列中没有找到值返回True，否则返回False。	x 不在 y序列中，如果x不在y序列中返回True。

• 小数运算

- 浮点数运行的问题：没有精确度

```
>>> 0.3+0.3+0.3+0.1
0.9999999999999999
>>> 0.3-0.1-0.1-0.1
-2.7755575615628914e-17
```

- 小数对象decimal：具有精确度
decimal 模块是python内置模块， 不需要安装，直接导入即可

```
import decimal
d = decimal.Decimal('3.1415926')
```

用字符串创建 Decimal 数：精确存储，**不需要、也不涉及精度设置**：最好传入一个字符串，如果传入了一个浮点型，decimal对象的浮点精度得不到保证。



精度设置：仅用于控制**算术运算结果**的有效数字位数

全局精度设置

```
decimal.getcontext().prec = 3
```

局部精度设置

```
with decimal.localcontext() as ctx:
```

```
ctx.prec = 3
```

```
import decimal
```

```
decimal.getcontext().prec = 3
```

```
a=decimal.Decimal(0.3)
```

```
b=decimal.Decimal(0.1)
```

```
a+a+a+b
```

如果你手动改精度，**只会影响运算结果，不影响你创建的数字**

```
import decimal
```

```
decimal.getcontext().prec = 3 # 手动设精度为3位有效数字
```

```
d = decimal.Decimal('3.1415926') # 依然精确存储，不受影响！
```

```
print(d) # 输出 3.1415926（原样存储）
```

```
print(d / 2) # 输出 1.57（运算结果按3位精度截断）
```

• 分数运算

- 分数对象 Fraction

Fraction是用于进行分数计算的模块

普通小数（比如 `0.1 + 0.2`）会有计算误差，而**分数运算 100% 精确**，这是 `fractions` 最大的用处。

```
# 从 fractions 模块中，导入 分数类 Fraction（专门处理分数运算）
from fractions import Fraction
```

```
# 创建分数 a: Fraction(分子，分母) → 5/4（四分之五）
```

```
a = Fraction(5, 4)
```

```
# 创建分数 b: 7/16（十六分之七）
```

```
b = Fraction(7, 16)
```

```
# 分数加法: 5/4 + 7/16
```

```
print(a + b)
```

```
# 分数乘法: 5/4 × 7/16
```

```
print(a * b)
```

```
# 把 a×b 的结果，赋值给变量 c
c = a * b

# 获取分数 c 的 分母 (denominator = 分母)
print(c.denominator)
# 获取分数 c 的 分子 (numerator = 分子)
print(c.numerator)
```

week2

变量

- 变量命名
 - 标识符可以由字母、数字、下划线 (_) 组成，其中数字不能打头。
 - 标识符不能是 Python 关键字，但可以包含关键字。
 - 标识符不能包含空格。
 - Python 还包含一系列关键字和内置函数，一般也不建议使用它们作为变量名：如果开发者尝试使用关键字作为变量名，Python 解释器会报错。如果开发者使用内置函数的名字作为变量名，Python 解释器不会报错，只是该内置函数就被这个变量覆盖了，该内置函数就不能使用了。

表 1 Python 关键字

False	None	True	and	as
assert	break	class	continue	def
del	elif	else	except	finally
for	from	global	if	import
in	is	lambda	nonlocal	not
or	pass	raise	return	try
while	with	yield		

表 2 Python内置函数

abs()	all()	any()	basestring()	bin()
bool()	bytearray()	callable()	chr()	classmethod()
cmp()	compile()	complex()	delattr()	dict()
dir()	divmod()	enumerate()	eval()	execfile()
file()	filter()	float()	format()	frozenset()
getattr()	globals()	hasattr()	hash()	help()
hex()	id()	input()	int()	isinstance()
issubclass()	iter()	len()	list()	locals()
long()	map()	max()	memoryview()	min()
next()	object()	oct()	open()	ord()
pow()	print()	property()	range()	raw_input()
reduce()	reload()	repr()	reversed()	zip()
round()	set()	setattr()	slice()	sorted()
staticmethod()	str()	sum()	super()	tuple()
type()	unichr()	unicode()	vars()	xrange()
Zip()	__import__()	apply()	buffer()	coerce()
intern				

 #查看关键字

import keyword

keyword.kwlist

变量与类型

语句

X=5

步骤

- 创建对象5
- 判断变量x是否存在
 - 不存在, 创建x
- 建立对象5和变量x的引用

- 在 Python 世界里，变量名（如 a, x）只是一个标签（名字），而数据本身是装在盒子里的（变量是一个盒子，数据直接存储在变量中❌）
所以盒子是什么类型，变量就是什么类型
- Python变量特点：
 - 变量在第一次赋值时被创建，再次出现时直接使用
 - 变量没有数据类型，Python是动态类型语言，变量不需要也不允许显式声明类型（变量声明时必须指定数据类型❌），数据类型属于对象，类型决定了对象在内存中的存储方式
 - 变量本质上是内存中对象的引用(或标签/名字)，数据本身存储在对象中。变量引用了对象。在表达式中使用变量时，变量立即被其引用的对象替代。所以变量在使

用前必须给其赋值。

- 多个变量可以同时引用同一个对象

数据类型

- **不可变类型**（可哈希）
 - 盒子密封、不可动
 - 对它们进行修改操作(如拼接、替换)实际上会创建一个新的对象
 - 如：数字 (int, float)、字符串 (str)、元组 (tuple)
- **可变类型**（不可哈希）
 - 盒子开放、可变动
 - 可以直接在原内存地址上修改其内容(如 append, remove)，无需创建新对象
 - 如：列表 (list)、字典 (dict)、集合 (set)

赋值语句

- 基础赋值 `a=10`

这是 Python 最基础的赋值方式，就是把右边的值 `10` 存到左边的变量 `a` 里。

```
a = 10
print(a) # 输出 10
```

- 元组赋值并行赋值 `(a,b = 10,20)`

本质是把右边的(10,20)这个元组，按位置依次拆包给左边的变量。(a,b) = 10,20写法是一样的，括号可以省略。

```
a, b = 10, 20
print(a) # 10
print(b) # 20
```

👉 这也是 Python 里交换变量最方便的方式：`a, b = b, a`，不用写中间变量。

- 列表赋值 `[a,b] = [10,20]`

和元组赋值逻辑几乎一样，只是左边用列表接收，右边也是列表。它的核心特点是**按位置一一对应**，位置不能乱，也不能多 / 少。

```
[a, b] = [10, 20]
print(a) # 10
print(b) # 20
```

- 序列赋值（通用性）`a,b = 'AB' / a,b = [10,20]`

这是 Python 的通用特性：只要右边是**可迭代的序列（字符串、列表、元组等）**，都可以按位置拆包赋值给左边的变量。

- 字符串例子：

```
a, b = 'AB'
print(a) # 'A' (字符串第1个字符)
print(b) # 'B' (字符串第2个字符)
```

- 列表例子：

```
a, b = [10, 20]
print(a) # 10 (列表第1个元素)
print(b) # 20 (列表第2个元素)
```

⚠ 注意：左边变量的数量，必须和右边序列的长度完全一致，否则会报错。

- 扩展的序列解包 `a,*b = 'hello'`（Python3 特有）

这是 Python3 的高级玩法，用 `*` 号来**收集剩下的所有元素**，不用再手动数数量。

```
a, *b = 'hello'
print(a) # 'h' (第一个字符)
print(b) # ['e', 'l', 'l', 'o'] (剩下的所有字符，打包成列表)
```

- `b` 会接收除了前面变量之外的所有剩余元素，组成一个列表
- 它可以放在任意位置：`a, b = 'hello' → a=['h','e','l','l'], b='o'`

- 多目标赋值 `a = b = c = 10`

一次性给多个变量赋同一个值，三个变量都会指向同一个值 `10`。

```
a = b = c = 10
print(a, b, c) # 10 10 10
```

⚠ 注意：如果右边是可变对象（比如列表），`a = b = []` 会让 `a` 和 `b` 指向同一个列表，修改其中一个会影响另一个。

- 增强赋值 `a += 1`

是简写形式，等价于 `a = a + 1`，把运算和赋值合并在一起。

```
a = 10
a += 1 # 等价于 a = a + 1
print(a) # 11
```

类似的还有 `--`、`*=`、`/=` 等，比如 `a *= 2` 就是 `a = a * 2`。

对象回收机制

- Python为每一个对象创建一个计数器，**记录对象的引用次数**。
- **当计数器为0**，对象没有任何引用时，其占用的内存空间会被自动回收
- 计数方式不能解决循环交叉引用，因此，python还有标记清除和分代收集的回收机制

```
a = []
b = []
a.append(b) # a 引用 b
b.append(a) # b 引用 a
```

循环引用（交叉引用）a 里面装着 b，b 里面装着 a，双方引用计数永远 ≥ 1 ，引用计数永远降不到 0

标记清除（专治循环引用）+ **分代收集**（优化效率）

共享引用

- 共享引用是指同多个变量引用了同一个对象

- 判断两个变量是否引用了同一个对象的关键字是 is
- 判断两个变量是否具有相同的值 ==

函数

- 内置函数
 - 数学函数

函数	返回值（描述）
abs(x)	返回数字的绝对值，如abs(-10) 返回 10
ceil(x)	返回数字的上入整数，如math.ceil(4.1) 返回 5
cmp(x, y)	如果 x < y 返回 -1, 如果 x == y 返回 0, 如果 x > y 返回 1
exp(x)	返回e的x次幂(e ^x),如math.exp(1) 返回2.718281828459045
fabs(x)	返回数字的绝对值，如math.fabs(-10) 返回10.0
floor(x)	返回数字的下舍整数，如math.floor(4.9)返回 4
log(x)	如math.log(math.e)返回1.0,math.log(100,10)返回2.0
log10(x)	返回以10为基数的x的对数，如math.log10(100)返回 2.0
max(x1, x2,...)	返回给定参数的最大值，参数可以为序列。
min(x1, x2,...)	返回给定参数的最小值，参数可以为序列。
modf(x)	返回x的整数部分与小数部分，两部分的数值符号与x相同，整数部分以浮点型表示。
pow(x, y)	x**y 运算后的值。
round(x [,n])	返回浮点数x的四舍五入值，如给出n值，则代表舍入到小数点后的位数。
sqrt(x)	返回数字x的平方根

- 类型转换函数

python常用类型转换函数		
函数格式	使用示例	描述
int(x [,base])	int("8")	可以转换的包括String类型和其他数字类型，但是会丢失精度
float(x)	float(1)或者float("1")	可以转换String和其他数字类型，不足的位数用0补齐，例如1会变成1.0
complex(real ,imag)	complex("1")或者complex(1,2)	第一个参数可以是String或者数字，第二个参数只能为数字类型，第二个参数没有时默认为0
str(x)	str(1)	将数字转化为String
repr(x)	repr(Object)	返回一个对象的String格式
eval(str)	eval("12+23")	执行一个字符串表达式，返回计算的结果,如例子中返回35
tuple(seq)	tuple((1,2,3,4))	参数可以是元组、列表或者字典，wie字典时，返回字典的key组成的集合
list(s)	list((1,2,3,4))	将序列转变成一个列表，参数可为元组、字典、列表，为字典时，返回字典的key组成的集合
set(s)	set(['b','r','u','o','n'])或者set("asdfg")	将一个可以迭代对象转变为可变集合，并且去重复，返回结果可以用来计算差集x - y、并集x y、交集x & y
frozenset(s)	frozenset([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])	将一个可迭代对象转变成不可变集合，参数为元组、字典、列表等。
chr(x)	chr(0x30)	chr()用一个范围在 range（256）内的（就是0～255）整数作参数，返回一个对应的字符。返回值是当前整数对应的ascii字符。
ord(x)	ord('a')	返回对应的 ASCII 数值，或者 Unicode 数值
hex(x)	hex(12)	把一个整数转换为十六进制字符串
oct(x)	oct(12)	把一个整数转换为八进制字符串

- math库函数 import math

1.基本函数

求最大整数	floor(1.9)=1
求最小整数	ceil(0.9)=1
整除运算	fmod(3,2)=1
求和	fsum(item)
求阶乘	factorial(n)
求最大的公约数	gcd(4,6)=2
求绝对值	fabs(num)
符号函数(把后数的符号给前数)	copysign(n1,n2)
分割整数与小数	modf(1.5)=0.5, 1

2.对数函数

e为底数	log(n)
其他数为底数	log(n, basic)
2为底数	log2(n)
10为底数	log10(n)
求次幂	pow(x,y)
求根号	sart(n)

3.三角函数

正弦函数	sin(x)
余弦函数	cos(x)
正切函数	tan(x)
反正弦函数	asin(x)
反余弦函数	acos(x)
反正切函数	atan(x)
欧几里得范数 (斜边)	hypot(3,4)=5

4.角度的切换

弧度转为角度	degrees(弧度)
角度转为弧度	radians(角度)

5.双曲函数

正弦双曲函数	sinh(x)
余弦双曲函数	cosh(x)
正切双曲函数	tanh(x)
反正弦双曲函数	asin(x)
反余弦双曲函数	acos(x)
反正切双曲函数	atanh(x)

6.math定义的常数

pi	$\pi = 3.141592 \dots$, 可用精度
e	$e = 2.718281 \dots$, 达到可用的精度
tau	$\tau = 6.283185 \dots$, 可达到精度
inf	无穷大
nan	非数字

■ import random

- random.randint(a,b)
生成一个int（整数）型的随机数,此处的a为产生随机数的左边界，b为右边界（包含端点）
- random.random()
可以生成0到1的一个随机浮点数
- random.randint(a,b)
生成一个float（浮点数）型的随机数,此处的a为产生随机数的左边界，b为右边界（包含端点）
- random.choice(seq)
用于从一个非空序列（如列表、元组、字符串等）中随机选择一个元素

集合

- 创建集合
 - 集合（set）是一个**无序的不重复、不可变类型元素组成**的整体。
可以使用大括号 `{ }` 或者 `set()` 函数创建集合



创建一个空集合必须用 **set()** 而不是 `{ }`，因为 `{ }` 是用来创建一个空字典。
集合中的元素必须是不可变类型（故集合的元素**不可以是列表、字典、集合**），否则会抛出TypeError
集合中的元素是无序的，**不支持索引访问**(如 `s[0]`)
集合中的元素必须是唯一的，自动去重

frozenset是冻结的集合，它是不可变的，好处是它可以作为字典的key，也可以作为其它集合的元素。缺点是一旦创建便不能更改，没有add，remove方法。

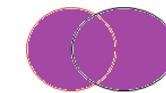
• 集合运算

集合之间也可进行数学集合运算（例如：并集、交集等），可用相应的操作符或方法来实现。

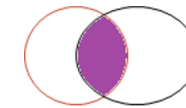
- 子集 `<,issubset()`



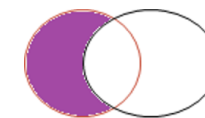
- 并集 `|,union()`



- 交集 `&,intersection()`



- 差集 `-,difference()`



- 对称差 `^,symmetric_difference()`



• 集合方法

- s.add() 向集合中添加元素
 - s.clear() 清空集合
 - s.copy() 返回集合的浅拷贝
 - s.pop() 删除并返回任意的集合元素（如果集合为空，会引发 KeyError）
 - s.remove() 删除集合中的一个元素（如果元素不存在，会引发 KeyError）
 - s.discard() 删除集合中的一个元素（如果元素不存在，则不执行任何操作，不会引发 keyError）
 - x.isdisjoint(y) 如果两个集合x,y有一个空交集，返回 True
 - x.update(y) 用自己x和另一个集合y的并集来更新这个集合x
 - x.intersection_update(y) 用自己x和另一个集合y的交集来更新这个集合x
 - x.difference_update(y) 从这个集合x中删除另一个集合y的所有元素来更新这个集合x
 - x.symmetric_difference_update(y) 用自己x和另一个y的对称差来更新这个集合x
- 集合内置函数

函数	描述
all()	如果集合中的所有元素都是 True（或者集合为空），则返回 True。
any()	如果集合中的所有元素都是 True，则返回 True；如果集合为空，则返回 False。
enumerate()	返回一个枚举对象，其中包含了集合中所有元素的索引和值（配对）。
len()	返回集合的长度（元素个数）
max()	返回集合中的最大项
min()	返回集合中的最小项
sorted()	从集合中的元素返回新的排序列表（不排序集合本身）
sum()	返回集合的所有元素之和

week4

列表

- 列表概念
 - 列表是Python中内置**有序、可变序列**，列表的所有元素放在一对中括号“[]”中，并使用逗号分隔；
 - 当列表元素增加或删除时，列表对象**自动进行扩展或收缩内存**，保证元素之间没有缝隙；

- 在Python中，一个列表中的**数据类型可以各不相同**，可以同时分别为整数、实数、字符串等基本类型，甚至是列表、元组、字典、集合以及其他自定义类型的对象。

[10, 20, 30, 40]

['crunchy frog', 'ram bladder', 'lark vomit']

['spam', 2.0, 5, [10, 20]]

[['file1', 200,7], ['file2', 260,9]]

- 列表方法

方法	说明
lst.append(x)	将元素x添加至列表lst尾部
lst.extend(L)	将列表L中所有元素添加至列表lst尾部
lst.insert(index, x)	在列表lst指定位置index处添加元素x，该位置后面的所有元素后移一个位置
lst.remove(x)	在列表lst中删除首次出现的指定元素，该元素之后的所有元素前移一个位置
lst.pop([index])	删除并返回列表lst中下标为index（默认为-1）的元素
lst.clear()	删除列表lst中所有元素，但保留列表对象
lst.index(x)	返回列表lst中第一个值为x的元素的下标，若不存在值为x的元素则抛出异常
lst.count(x)	返回指定元素x在列表lst中的出现次数
lst.reverse()	对列表lst所有元素进行逆序
lst.sort(key=None, reverse=False)	对列表lst中的元素进行排序，key用来指定排序依据（给一个函数，比如len），reverse决定升序（False），还是降序（True）
lst.copy()	返回列表lst的浅复制

- 列表的创建与删除

- 使用"="直接将一个列表赋值给变量即可创建列表对象
>>> a_list = ['a', 'b', 'mpilgrim', 'z', 'example']
>>> a_list = [] #创建空列表
 - 也可以使用list()函数将元组、range对象、字符串或其他类型的可迭代对象类型的数据转换为列表。
>>> a_list = list((3,5,7,9,11))
>>> a_list
[3, 5, 7, 9, 11]
>>> list(range(1,10,2))
[1, 3, 5, 7, 9]
>>> list('hello world')
['h', 'e', 'l', 'l', 'o', ' ', 'w', 'o', 'r', 'l', 'd']
>>> x = list() #创建空列表
 - 当不再使用时，使用del命令删除整个列表，如果列表对象所指向的值不再有其他对象指向，Python将同时删除该值。
>>> del a_list
- 列表元素的增加

- 可以使用"+"运算符将元素添加到列表中。
>>> aList = [3,4,5]
>>> aList = aList + [7]
>>> aList
[3, 4, 5, 7]
严格意义上讲，这并不是真的为列表添加元素，而是创建了一个新列表，并将原列表中的元素和新元素依次复制到新列表的内存空间。由于涉及大量元素的复制，该操作速度较慢，在涉及大量元素添加时不建议使用该方法。
- 使用列表对象的append()方法在当前列表尾部追加元素，原地修改列表，是真正意义上的在列表尾部添加元素，速度较快。
>>> aList.append(9)
>>> aList
[3, 4, 5, 7, 9]
所谓"原地"，是指不改变列表在内存中的首地址。

3、使用列表对象的extend()方法可以将另一个迭代对象的所有元素添加至该列表对象尾部。通过extend()方法来增加列表元素也不改变其内存首地址，属于原地操作。

```
>>> a.extend([7,8,9])  
>>> a  
[5, 2, 4, 7, 8, 9]  
>>> id(a) #返回内存地址  
25289752  
>>> aList.extend([11,13])  
>>> aList  
[3, 4, 5, 7, 9, 11, 13]  
>>> aList.extend((15,17))  
>>> aList  
[3, 4, 5, 7, 9, 11, 13, 15, 17]
```

- 使用列表对象的insert()方法将元素添加至列表的指定位置。
>>> aList.insert(3, 6) #在下标为3的位置插入元素6
>>> aList
[3, 4, 5, 6, 7, 9, 11, 13, 15, 17]
应尽量从列表尾部进行元素的增加与删除操作。
列表的insert()可以在列表的任意位置插入元素，但由于列表的自动内存管理功能，insert()方法会引起插入位置之后所有元素的移动，会影响处理速度。

append 运行速度明显优于insert

- 使用乘法来扩展列表对象，将列表与整数相乘，生成一个新列表，新列表是原列表中元素的重复。
>>> aList = [3,5,7]
>>> bList = aList
>>> id(aList)
57091464
>>> id(bList)
57091464
>>> aList = aList*3
>>> aList
[3, 5, 7, 3, 5, 7, 3, 5, 7]
>>> bList
[3,5,7]
>>> id(aList)
57092680
>>> id(bList)
57091464

当使用*运算符将包含列表的列表重复并创建新列表时，并不是复制子列表值，而是复制子序列已有元素的引用。因此，当修改其中一个值时，相应的引用也会被修改。

```
>>> x = [[None] * 2] * 3  
>>> x  
[[None, None], [None, None], [None, None]]  
>>> x[0][0] = 5  
>>> x  
[[5, None], [5, None], [5, None]]  
>>> x = [[1,2,3]] * 3  
>>> x[0][0] = 10  
>>> x  
[[10, 2, 3], [10, 2, 3], [10, 2, 3]]
```

- 列表元素的删除

```

1.使用del命令删除列表中的指定位置上的元素。
>>> a_list = [3,5,7,9,11]
>>> del a_list[1]
>>> a_list
[3, 7, 9, 11]
2.使用列表的pop()方法删除并返回指定位置（默认为最后一个）上的元素，如果给定的索引超出了列表的范围则抛出异常。
>>> a_list = list((3,5,7,9,11))
>>> a_list.pop()
11
>>> a_list
[3, 5, 7, 9]
>>> a_list.pop(1)
5
>>> a_list
[3, 7, 9]

```

```

3、使用列表对象的remove()方法删除首次出现的指定元素，如果列表中不存在要删除的元素，则抛出异常。
>>> a_list = [3,5,7,9,7,11]
>>> a_list.remove(7)
>>> a_list
[3, 5, 9, 7, 11]

```

把列表中的1删除。

分析以下语句错在哪里？

```

x = [1, 2, 1, 2, 1, 2, 1, 1, 2]
for i in x:
    if i == 1:
        x.remove(i)
print(x)

```

[2, 2, 2, 1, 2]

在删除列表元素时，Python会自动对列表内存进行收缩并移动列表元素以保证所有元素之间没有空隙，增加列表元素时也会自动扩展内存并对元素进行移动以保证元素之间没有空隙。每当插入或删除一个元素之后，该元素位置后面所有元素的索引就都改变了。

正确的代码：

```

>>> x = [1,2,1,2,1,1,1]
>>> for i in x[:]:
    #切片 x[:] 是一个不同于x新对象
    if i == 1:
        x.remove(i)

```

或者：

```

>>> x = [1,2,1,2,1,1,1]
>>> for i in range(len(x)-1,-1,-1):
    #从后往前删
    if x[i]==1:
        del x[i]

```

• 列表元素访问与计数

- 使用下标直接访问列表元素，如果指定下标不存在，则抛出异常

列表支持使用负索引访问元素

- 使用列表对象的index()方法获取指定元素首次出现的下标，若列表对象中不存在指定元素，则抛出异常
- 使用列表对象的count()方法统计指定元素在列表对象中出现的次数

• 成员资格判断

使用in关键字来判断一个值是否存在于列表中，返回结果为“True”或“False”

• 切片操作

- 使用切片来截取列表中的任何部分，得到一个新列表，也可以通过切片来修改和删除列表中部分元素，甚至可以通过切片操作为列表对象增加元素。
- 切片适用于列表、元组、字符串、range对象等类型，但作用于列表时功能最强大。

- 切片使用2个冒号分隔的3个数字来完成：第一个数字表示切片开始位置（默认为0）。第二个数字表示切片截止位置（但不包含该数字的索引位置）（默认为列表长度）。第三个数字表示切片的步长（默认为1），当步长省略时可以顺便省略最后一个冒号。
- 切片操作不会因为下标越界而抛出异常，而是简单地在列表尾部截断或者返回一个空列表，代码具有更强的健壮性。

```

aList = [3, 4, 5, 6, 7, 9, 11, 13, 15, 17]
print(aList[:])      #返回包含所有元素的新列表
print(aList[::-1])   #逆序的所有元素
print(aList[::2])     #偶数位置，隔一个取一个
print(aList[1::2])    #奇数位置，隔一个取一个
print(aList[3:])      #从下标3开始的所有元素
print(aList[3:6])     #下标在[3, 6)之间的所有元素
print(aList[0:100:1]) #前100个元素，自动截断
print(aList[100:])    #下标100之后的所有元素，自动截断

```



```

[3, 4, 5, 6, 7, 9, 11, 13, 15, 17]
[17, 15, 13, 11, 9, 7, 6, 5, 4, 3]
[3, 5, 7, 11, 15]
[4, 6, 9, 13, 17]
[6, 7, 9, 11, 13, 15, 17]
[6, 7, 9]
[3, 4, 5, 6, 7, 9, 11, 13, 15, 17]
[]

```

使用切片来原修改列表内容

```

aList = [3, 5, 7]
aList[len(aList):] = [9] #在尾部追加元素
print(aList)
[3, 5, 7, 9]
aList[:3] = [1, 2, 3] #替换前3个元素
print(aList)
aList[:3] = [] #删除前3个元素
print(aList)
aList = list(range(10))
print(aList)
aList[::2] = [0]*5 #替换偶数位置上的元素
print(aList)

```

```

[3, 5, 7, 9]
[1, 2, 3, 9]
[9]
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
[0, 1, 0, 3, 0, 5, 0, 7, 0, 9]

```

使用del与切片结合来删除列表元素

```

aList = [3,5,7,9,11]
del aList[:3] #删除前3个元素
print(aList)

aList = [3,5,7,9,11]
del aList[::2] #删除偶数位置上的元素
print(aList)

```

```

[9, 11]
[5, 9]

```



切片返回的是列表元素的浅复制

所谓浅复制，是指生成一个新的列表，并且把原列表中所有元素的引用都复制到新列表中。

如果原列表中只包含整数、实数、复数等基本类型或元组、字符串这样的不可变类型的数据，一般是没问题的。

如果原列表中包含列表之类的可变数据类型，由于浅复制时只是把子列表的引用复制到新列表中，这样的话修改任何一个都会影响另外一个。

```
aList = [3, 5, 7]
bList = aList[:] #切片，浅复制
print(aList == bList) #两个列表的元素完全一样
print(aList is bList) #但不是同一个对象

bList[1] = 8 #修改其中一个不会影响另一个
print(bList)
print(aList)

x = [1, 2, [3, 4]]
y = x[:]
x[0] = 5
print(x)
print(y)

x[2].append(6)
print(x)
print(y)
```



```
True
False
[3, 8, 7]
[3, 5, 7]
[5, 2, [3, 4]]
[1, 2, [3, 4]]
[5, 2, [3, 4, 6]]
[1, 2, [3, 4, 6]]
```

列表排序

- 使用列表对象的sort()方法进行原地排序，支持多种不同的排序方法。
- 使用内置函数sorted()对列表进行排序并返回新列表
- 使用列表对象的reverse()方法将元素原地逆序
- 使用内置函数reversed()对列表元素进行逆序排列并返回迭代对象

```
aList = [3, 4, 5, 6, 7, 9, 11, 13, 15, 17]
import random
random.shuffle(aList)
print(aList)

aList.sort() #默认是升序排序
print(aList)
aList.sort(reverse = True) #降序排序
print(aList)

#按转换成字符串的长度排序
aList.sort(key = lambda x:len(str(x))) #lambda匿名函数 输出为输入字符串的长度
print(aList)

newList=list(sorted(aList)) #升序排序
print(aList)
newList=list(sorted(aList,reverse = True))
print(newList)

aList = [3, 4, 5, 6, 7, 9, 11, 13, 15, 17]
aList.reverse()
print(aList)

newList = list(reversed(aList)) #返回reversed对象
print(newList)
```



```
[17, 5, 7, 3, 13, 9, 11, 15, 4, 6]
[3, 4, 5, 6, 7, 9, 11, 13, 15, 17]
[17, 15, 13, 11, 9, 7, 6, 5, 4, 3]
[9, 7, 6, 5, 4, 3, 17, 15, 13, 11]
[9, 7, 6, 5, 4, 3, 17, 15, 13, 11]
[17, 15, 13, 11, 9, 7, 6, 5, 4, 3]
[17, 15, 13, 11, 9, 7, 6, 5, 4, 3]
[3, 4, 5, 6, 7, 9, 11, 13, 15, 17]
```



匿名函数lambda

一类无需定义标识符（函数名）的函数或子程序

可以接收任意多个参数（包括可选参数）并且返回单个表达式的值

lambda [arg1 [,arg2,.....argn]]:expression

s=lambda x,y:x+y

h=lambda x,y,z:x+y-z

print(s(10,20),h(10,20,50))

sorted()和list.sort()对比

对比维度	sorted()	list.sort()
类型	内置全局函数	列表专属方法
适用范围	所有可迭代对象（列表、元组、字符串、集合、字典等）	仅列表可用
是否修改原数据	✗ 不修改原数据	✓ 原地修改原列表
结果形式	返回全新排序后的列表	无返回值，返回 None
内存 / 效率	生成新副本，开销大，速度稍慢	原地排序，不创建副本， 更快、更省内存
使用场景	需要保留原数据、给非列表数据排序	不需要原列表，追求高效排序

用于序列操作的常用内置函数

- len(列表)：返回列表中的元素个数，同样适用于元组、字典、集合、字符串等。
- max(列表)、min(列表)：返回列表中的最大或最小元素，同样适用于元组、字典、集合、range对象等。
- sum(列表)：对列表的元素进行求和运算，对非数值型列表运算需要指定start参数，同样适用于元组、range。
- zip()函数：返回可迭代的zip对象（可迭代对象），这样就可以使用一个循环并行遍历两个列表
- enumerate(列表)：枚举列表元素，返回枚举对象，其中每个元素为包含下标和值的元组。该函数对元组、字符串同样有效。返回迭代器，不是直接返回列表

变量共享引用

判断以下几段语句的输出

```
A="Spamming"
B=A
B="Shrubbery"
print(A,B)
```

Spamming Shrubbery

A和B最初共享与引用，但是后来B重新指向了另一个对象，所以不影响A

```
A=["Spamming"]
B=A
B[0]="Shrubbery"
print(A,B)
```

['Shrubbery'] ['Shrubbery']

A和B最初共享引用，但是列表支持在原处修改，所以赋值B[0]就是在原处修改了，那么A肯定也更改了，区别就在于B[0]就是在原处修改。

```
A=["Spamming"]
B=A[:]
B[0]="Shrubbery"
print(A,B)
```

['Spamming'] ['Shrubbery']

A和B最初共享引用，A[:]会新生成一个新对象，所以赋值B[0]重新指向别的地方指针，那么A不会受影响。

元组

- 元组(tuple)
同列表类似，不同的地方在于，元组里的元素是不能更改的
元组使用圆括号，列表使用方括号。
 - 创建元组
tup1 = ('physics', 'chemistry', 1997, 2000) #数据类型可以各不相同
tup2 = (1, 2, 3, 4, 5)
tup3 = "a", "b", "c", "d"
tup4 = ()#空元组
tup5 = (50,) #元组中只包含一个元素时，需要在元素后面添加逗号
 - 元组连接 +
tup6 = tup1 + tup2
 - 元组删除 -
del tup1 #整个元组删除
- 元组运算符

Python 表达式	结果	描述
len((1, 2, 3))	3	计算元素个数
(1, 2, 3) + (4, 5, 6)	(1, 2, 3, 4, 5, 6)	连接
('Hi!') * 4	('Hi!', 'Hi!', 'Hi!', 'Hi!')	复制
3 in (1, 2, 3)	True	元素是否存在
for x in (1, 2, 3): print x,	1 2 3	迭代

- 元组索引与截取

```
L = ('spam', 'Spam', 'SPAM!')
```

Python 表达式	结果	描述
L[2]	'SPAM!'	读取第三个元素
L[-2]	'Spam'	反向读取，读取倒数第二个元素
L[1:]	('Spam', 'SPAM!')	截取元素

- 内置函数

序号	方法及描述
1	<code>cmp(tuple1, tuple2)</code> 比较两个元组元素。
2	<code>len(tuple)</code> 计算元组元素个数。
3	<code>max(tuple)</code> 返回元组中元素最大值。
4	<code>min(tuple)</code> 返回元组中元素最小值。
5	<code>tuple(seq)</code> 将列表转换为元组。



```
t1 = (100)
t2 = (100,)
```

t1 是 int (整数)，t2 是 tuple (元组)。t1的括号是数学分组括号，t2的括号是元组容器符号，元组中只包含一个元素时，需要在元素后面添加逗号。



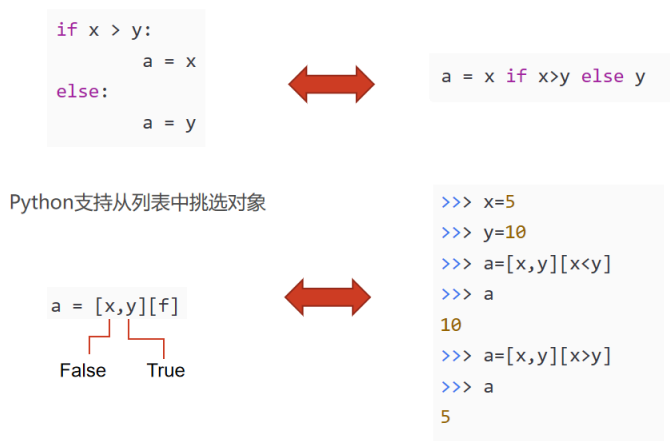
```
my_tuple = ('apple', 'banana', 'cherry')
my_tuple[1] = 'orange'
```

执行 my_tuple[1] = 'orange' 程序会报错（TypeError），因为元组里的元素是不能更改的（不可变序列）

条件语句

- 基本语法
if 判断条件：
 执行语句.....
else：
 执行语句.....

- python 并不支持 switch 语句，所以多个条件判断，只能用 elif 来实现，如果判断需要多个条件需同时判断时，可以使用 or （或），表示两个条件有一个成立时判断条件成功；使用 and （与）时，表示只有两个条件同时成立的情况下，判断条件才成功
- 三元表达式



week4

字符串

- 字符串表达与访问
 - 字符串是 Python 中最常用的数据类型。我们可以使用引号 ' 或 " 来创建字符串
 - Python 不支持单字符类型，单字符在 Python 中也是作为一个字符串使用。
 - Python 访问子字符串，可以使用方括号[]来截取字符串

```
var1 = 'Hello World!'
var2 = "Runoob"
print ("var1[0]: ", var1[0])    #var1[0]: H
print ("var2[1:5]: ", var2[1:5])    #var2[1:5]: unoo

var1 = 'Hello World!'
print ("已更新字符串 : ", var1[:6] + 'Runoo
b!')
#Hello Runoob!
```

转义字符	描述
\\(在行尾时)	续行符
\\	反斜杠符号
'	单引号
"	双引号

\a	响铃
\b	退格(Backspace)
\000	空
\n	换行
\v	纵向制表符
\t	横向制表符
\r	回车
\f	换页
\oyy	八进制数，yy 代表的字符，例如：\o12 代表换行，其中 o 是字母，不是数字 0。
\xyy	十六进制数，yy代表的字符，例如：\x0a代表换行
\other	其它的字符以普通格式输出

- 字符串跨越多行
 - python三引号允许一个字符串跨多行，字符串中可以包含换行符、制表符以及其他特殊字符

```
para_str = """这是一个多行字符串的实例
多行字符串可以使用制表符
TAB ( \t )。
也可以使用换行符 [ \n ]。
"""
print (para_str)
```

这是一个多行字符串的实例
多行字符串可以使用制表符
TAB ()。
也可以使用换行符 []。

- raw字符串
 - 表示原始字符串，所有字符都不进行转义

```
: print(r'\\(\\_)/ \\(_)/')
\\(_)/ \\(_)/
```

- 字符串运算符

操作符	描述	实例
+	字符串连接	a + b 输出结果： HelloPython

*	重复输出字符串	a*2 输出结果: HelloHello
[]	通过索引获取字符串中字符	a[1] 输出结果 e
[:]	截取字符串中的一部分, 遵循左闭右开原则, str[0,2] 是不包含第 3 个字符的。	a[1:4] 输出结果 ell
in	成员运算符 - 如果字符串中包含给定的字符返回 True	'H' in a 输出结果 True
not in	成员运算符 - 如果字符串中不包含给定的字符返回 True	'M' not in a 输出结果 True
r/R	原始字符串 - 原始字符串: 所有的字符串都是直接按照字面的意思来使用, 没有转义特殊或不能打印的字符。 原始字符串除在字符串的第一个引号前加上字母 r (可以大小写) 以外, 与普通字符串有着几乎完全相同的语法。	print(r'\n') print(R'\n')
%	格式字符串	

#字符串运算符

```
a = "Hello"
b = "Python"

print("a + b 输出结果: ", a + b)
print("a * 2 输出结果: ", a * 2)
print("a[1] 输出结果: ", a[1])
print("a[1:4] 输出结果: ", a[1:4])

if( "H" in a ):
    print("H 在变量 a 中")
else:
    print("H 不在变量 a 中")

if( "M" not in a ):
    print("M 不在变量 a 中")
else:
    print("M 在变量 a 中")

print( r'\n' )
print( R'\n' )
```

a + b 输出结果: HelloPython
a * 2 输出结果: HelloHello
a[1] 输出结果: e
a[1:4] 输出结果: ell
H 在变量 a 中
M 不在变量 a 中
\n
\n

• 字符串格式化

字符串格式化主要有三种方式: 传统的 % 操作符、.format() 方法和 f-string (格式化字符串字面值)。

◦ .format()方法

“模式字符串”.format(参数1, 参数2, ...)

▪ 按位置填充

```
name = "Alice"
age = 25 # {} 会按顺序替换为参数
text = "Hello, {}. You are {} years old.".format(name, age)
print(text) # 输出: Hello, Alice. You are 25 years old.
```

```
# 也可以指定索引顺序
text2 = "You are {1}. Hello, {0}.".format(name, age)
print(text2) # 输出: You are 25. Hello, Alice.
```

▪ 按名字填充

```
text = "Hello, {name}. You are {age}.".format(name="Bob", age=30)
print(text) # 输出: Hello, Bob. You are 30. # 甚至可以直接传入字典
data = {"name": "Charlie", "age": 35} text2 = "Hello, {name}. You are {age}.".format(**data)
print(text2) # 输出: Hello, Charlie. You are 35.
```

format格式控制 (对齐、精度、类型)

```
num = 3.1415926 # 保留两位小数
print("Pi is {:.2f}".format(num)) # 输出: Pi is 3.14
print("Number: {:>10}".format(123))
# 输出: Number: 123
# 宽度为10, 左对齐
print("Number: {:<10}".format(123))
# 输出: Number: 123
# 宽度为10, 居中对齐, 用*填充
print("Number: {:*^10}".format(123))
# 输出: Number: ***123***
# 百分比格式
ratio = 0.856
print("Success rate: {:.2%}".format(ratio))
# 输出: Success rate: 85.60%
```

◦ f-string

python3.6+引入的特性, 是目前最推荐、最简洁、性能最好的方式

在字符串引号前加上字母f或F, 直接在{}中写变量名或表达式

```
name = "David"
age = 28
height = 1.75
# 直接嵌入变量
text = f"Hello, {name}. You are {age}."
```

```
print(text)
# 输出: Hello, David. You are 28.
```

支持任意表达式

```
x = 10
y = 20
print(f"The sum of {x} and {y} is {x + y}.") # 输出: The sum of 10 and 20 is 30.
# 调用方法
name = "alice" print(f"Clean name: [{name.strip()}]") # 输出: Clean name: [alice]
# 条件判断
value = 15 print(f"The value is {'high' if value > 10 else 'low'}.") # 输出: The value is high.
```

语法: {变量:格式说明符}

```
python
1 num = 3.1415926
2
3 # 保留两位小数
4 print(f"Pi is {num:.2f}")
5 # 输出: Pi is 3.14
6
7 # 对齐与填充
8 print(f"Number: {123:>10}") # 右对齐
9 print(f"Number: {123:*^10}") # 居中, *填充
10
11 # 显示十六进制
12 val = 255
13 print(f"Hex: {val:x}") # 小写 hex -> ff
14 print(f"Hex: {val:X}") # 大写 hex -> FF
15
16 # 千位分隔符
17 big_num = 1000000
18 print(f"Money: ${big_num:,}")
19 # 输出: Money: $1,000,000
```

C. 调试神器 (Python 3.8+)

如果你在调试代码, 想打印变量名和值, 可以使用 `=` 符号。

```
python
1 user = "Eve"
2 score = 95
3
4 # 自动输出 "变量名=变量值"
5 print(f"{user=}, {score=}")
6 # 输出: user='Eve', score=95
```

。 %格式化

符号	描述
%c	格式化字符及其ASCII码
%s	格式化字符串
%d	格式化整数
%u	格式化无符号整型
%o	格式化无符号八进制数
%x	格式化无符号十六进制数
%X	格式化无符号十六进制数（大写）
%f	格式化浮点数字，可指定小数点后的精度
%e	用科学计数法格式化浮点数
%E	作用同%e，用科学计数法格式化浮点数
%g	%f和%e的简写
%G	%f 和 %E 的简写
%p	用十六进制数格式化变量的地址

符号	功能
*	定义宽度或者小数点精度
-	用做左对齐
+	在正数前面显示加号(+)
<sp>	在正数前面显示空格
#	在八进制数前面显示零('0')，在十六进制前面显示'0x'或者'0X'(取决于用的是'x'还是'X')
0	显示的数字前面填充'0'而不是默认的空格
%	'%%'输出一个单一的'%'
(var)	映射变量(字典参数)

m.n.	m 是显示的最小总宽度,n 是小数点后的位数(如果可用的话)
------	--------------------------------

• 字符串内置函数

capitalize()	将字符串的第一个字符转换为大写
center(width, fillchar)	返回一个指定的宽度 width 居中的字符串, fillchar 为填充的字符, 默认为空格。
count(str, beg= 0,end=len(string))	返回 str 在 string 里面出现的次数, 如果 beg 或者 end 指定则返回指定范围内 str 出现的次数
bytes.decode(encoding="utf-8", errors="strict")	解码给定的 bytes 对象, 这个 bytes 对象可以由 str.encode() 来编码返回。
encode(encoding='UTF-8',errors='strict')	以 encoding 指定的编码格式编码字符串, 如果出错默认报一个 ValueError 的异常, 除非 errors 指定的是'ignore'或者'replace'
endswith(suffix, beg=0, end=len(string))	检查字符串是否以 obj 结束, 如果 beg 或者 end 指定则检查指定的范围内是否以 obj 结束, 如果是, 返回 True,否则返回 False.
expandtabs(tabsize=8)	把字符串 string 中的 tab 符号转为空格, tab 符号默认的空格数是 8 。
find(str, beg=0, end=len(string))	检测 str 是否包含在字符串中, 如果指定范围 beg 和 end , 则检查是否包含在指定范围内, 如果包含返回开始的索引值, 否则返回-1
index(str, beg=0, end=len(string))	跟find()方法一样, 只不过如果str不在字符串中会报一个异常.
isalnum()	如果字符串至少有一个字符并且所有字符都是数字则返 回 True,否则返回 False
isalpha()	如果字符串至少有一个字符并且所有字符都是字母则返回 True, 否则返回 False
isdigit()	如果字符串只包含数字则返回 True 否则返回 False..
islower()	如果字符串中包含至少一个区分大小写的字符, 并且所有这些(区分大小写的)字符都是小写, 则返回 True, 否则返回 False

isnumeric()	如果字符串中只包含数字字符, 且至少有一个字符, 则返回 True, 否则返回 False
isspace()	如果字符串中只包含空白, 则返回 True, 否则返回 False.
istitle()	如果字符串是标题化的(见 title())则返回 True, 否则返回 False
isupper()	如果字符串中包含至少一个区分大小写的字符, 并且所有这些(区分大小写的)字符都是大写, 则返回 True, 否则返回 False
join(seq)	以指定字符串作为分隔符, 将 seq 中所有的元素(的字符串表示)合并为一个新的字符串
len(string)	返回字符串长度
ljust(width[, fillchar])	返回一个原字符串左对齐,并使用 fillchar 填充至长度 width 的新字符串, fillchar 默认为空格。

lower()	转换字符串中所有大写字符为小写.
lstrip()	截掉字符串左边的空格或指定字符。
maketrans()	创建字符映射的转换表, 对于接受两个参数的最简单的调用方式, 第一个参数是字符串, 表示需要转换的字符, 第二个参数也是字符串表示转换的目标。
max(str)	返回字符串 str 中最大的字母。
min(str)	返回字符串 str 中最小的字母。
replace(old, new [, max])	把 将字符串中的 str1 替换成 str2,如果 max 指定, 则替换不超过 max 次。
rfind(str, beg=0,end=len(string))	类似于 find()函数, 不过是从右边开始查找.
rindex(str, beg=0, end=len(string))	类似于 index(), 不过是从右边开始.
rjust(width,[, fillchar])	返回一个原字符串右对齐,并使用fillchar(默认空格) 填充至长度 width 的新字符串
rstrip()	删除字符串字符串末尾的空格.
split(str="", num=string.count(str))	num=string.count(str)) 以 str 为分隔符截取字符串, 如果 num 有指定值, 则仅截取 num+1 个子字符串
splitlines([keepends])	按照行('\r', '\r\n', \n')分隔, 返回一个包含各行作为元素的列表, 如果参数 keepends 为 False, 不包含换行符, 如果为 True, 则保留换行符。

startswith(substr, beg=0,end=len(string))	检查字符串是否是以指定子字符串 substr 开头，是则返回 True，否则返回 False。如果 beg 和 end 指定值，则在指定范围内检查。
strip([chars])	在字符串上执行 lstrip()和 rstrip()
swapcase()	将字符串中大写转换为小写，小写转换为大写
title()	返回"标题化"的字符串,就是说所有单词都是以大写开始，其余字母均为小写(见 istitle())
translate(table, deletechars="")	根据 str 给出的表(包含 256 个字符)转换 string 的字符,要过滤掉的字符放到 deletechars 参数中
upper()	转换字符串中的小写字母为大写
zfill (width)	返回长度为 width 的字符串，原字符串右对齐，前面填充0
isdecimal()	检查字符串是否只包含十进制字符，如果是返回 true，否则返回 false。

字典dict

基本概念

字典是一种通过名字或者关键字引用的数据结构，其键可以是数字、字符串、元组，这种结构类型也称之为映射

字典是另一种可变量容器模型，且可存储任意类型对象。

字典的每个键值(key: value)对用冒号(:)分割，每个对之间用逗号(,)分割，整个字典包括在花括号({})中,格式如下所示：**d = {key1: value1, key2: value2 }**

键必须是唯一的，但值则不必。

值可以取任何数据类型，但键必须是不可变的，如字符串，数字或元组。

```
dict = {'Alice': '2341', 'Beth': '9102', 'Cecil': '3258'}
dict2 = { 'abc': 123, 98.6: 37 }
print (" dict1['Alice']: ", dict1['Alice'])

dict1['Alice'] = '1234' # 更新 Age
dict2['School'] = "SILC" # 添加信息

del dict1['Alice'] # 删除键 'Name'
dict2.clear() # 清空字典
del dict2 # 删除字典
```

基本操作

- (1) len() : 返回字典中 键:值对 的数量
- (2) d[k] : 返回关键字对应的值;

- (3) d[k] = v : 将值关联到键值k上;
- (4) del d[k] :删除键值为k的项;
- (5) key in d : 键值key是否在d中，是返回True，否则返回False

- 字典的键(key)必须不可变(如：字符串，数字，元组)，值(key)可以为任意数据类型
- 字典虽然逻辑是无序的，但是从3.7版本开始每次执行打印顺序都一样。因此，并**不能通过位置进行访问**
- 利用For循环，根据键(key)或值(value)取出对应的值(value)或键(key)

```
d4 = {'Year': 2018, 'Month': 3, 'Day': 18}

# 循环所有的key
for item in d4:
    print(item)

# 循环所有的value
for item in d4.values():
    print(item)

# 循环所有的key:value
for item in d4.keys():
    print(item, ': ', d4[item])

# 用items方法 k接收key ,v接收value
for k, v in d4.items():
    print(k, v)
```

Year
Month
Day
2018
3
18
Year : 2018
Month : 3
Day : 18
Year 2018
Month 3
Day 18

函数方法

- str(dict) #输出字典可以打印的字符串表示

```
d6 = {'Year': 2018, 'Month': 3, 'Day': 18}
content = str(d6)
print(content)

{'Year': 2018, 'Month': 3, 'Day': 18}
```

- len(dict) #计算字典内元素的个数，即键的总数

```
d7 = {'Year': 2018, 'Month': 3, 'Day': 18}
count = len(d7)
print(count)

3
```

- type(variable) #返回输入的变量类型，如果变量类型是字典就返回字典

```
d8 = {'Year': 2018, 'Month': 3, 'Day': 18}
ty = type(d8)
print(ty)

<class 'dict'>
```

- copy (浅拷贝)

```
d4 = {'Year': 2018, 'Month': 3, 'Day': 18}
dict2 = d4.copy()
print("新复制的字典为：", dict2)

新复制的字典为： {'Year': 2018, 'Month': 3, 'Day': 18}
```



浅拷贝

1. 你修改的是「外层」或「不可变元素」→ 不会影响原对象

比如你往新列表里加元素、删元素，或者修改里面的不可变对象（数字、字符串、元组）

✓ 这两种修改，都只作用在新的外层容器上，原对象完全不受影响。

2. 你修改的是「内层的可变对象」→ 会影响原对象！

如果列表里还嵌套了列表 / 字典这种可变对象，你通过新列表修改内层对象时，原对象里的也会跟着变

✗ 为什么会这样？因为 `b[2]` 和 `a[2]` 指向的是同一个内层列表，你改的是这个列表本身，两个引用都会看到变化。

- fromkeys() 创建一个新字典

`fromkeys(args, **kwargs)` 以args中元素做字典的键，**kwargs为字典所有键对应的初始值

```
new_d = ('Year', 'Month', 'Day')
dict1 = dict.fromkeys(new_d)
print("新的字典：%s" % str(dict1))

dict1 = dict.fromkeys(new_d, 8)
print("新的字典：%s" % str(dict1))

新的字典： {'Year': None, 'Month': None, 'Day': None}
新的字典： {'Year': 8, 'Month': 8, 'Day': 8}
```

- get()方法

根据key获取值 `get(self, k, d=None)` 参数：k:待操作的键(key), d:默认为None，在获取键的时候，如果不存在就返回 None 或指定什么就返回什么，**不报错**

```
d9 = {'Year': 2018, 'Month': 3, 'Day': 18}
v = d9.get('Day')
print('key存在时:', v)
v1 = d9.get('Hour')
print('key不存在时:', v1)
v2 = d9.get('Hour', 1234)
print("key不存在时并指定返回值:", v2)
```

key存在时: 18
key不存在时: None
key不存在时并指定返回值: 1234

- pop()方法

删除指定的key，`pop(self, k, d=None)` 参数：k:待删除的key d:如果不存在待删除的key，则返回指定值None，存在的话，返回对应的value

没有d=None，key不存在会报错

键被删除，对应值会被返回，可以存在遍历里，字典里就没有这个键值对了

```
d9 = {'Year': 2018, 'Month': 3, 'Day': 18}
v = d9.pop('Day')
print('key存在时:', v) # key存在时: 18

v2 = d9.pop('Hour', 1234)
print("key不存在时并指定返回值:", v2) # key不存在时并指定返回值: 1234

v1 = d9.pop('Hour')
print('key不存在时:', v1) # key不存在时报错
```

key存在时: 18
key不存在时并指定返回值: 1234

```
KeyError                                Traceback (most recent call last)
<ipython-input-8-d1b00b241c2c> in <module>
      6 print("key不存在时并指定返回值:", v2) # key不存在时并指定返回值: 1234
      7
----> 8 v1 = d9.pop('Hour')
      9 print('key不存在时:', v1) # key不存在时报错

KeyError: 'Hour'
```

- popitem()方法，随机删除一个键值对，并返回键值对

```
d10 = {'Year': 2018, 'Month': 3, 'Day': 18}
v1 = d10.popitem()
k, v = d10.popitem()
print(v1, k, v)
```

('Day', 18) Month 3

- setdefault 方法

给字典设置(增加)新的键，存在就不设置(获取当前key对应的值)，不存在就设置并返回当前key对应的值 `setdefault(self, k, d=None)` 参数：k:待设置(添加)的key d:默认为空，可以指定value值

```
d11 = {'Year': 2018, 'Month': 3, 'Day': 18}
v = d11.setdefault('hour', '10') # 如果待加入的key跟字典内重复，则不会添加
print(d11, v)
```

```
{ 'Year': 2018, 'Month': 3, 'Day': 18, 'hour': '10' } 10
```

- update方法，更新字典 update(self, E=None, **F)

```
d12 = { 'Year': 2018, 'Month': 3, 'Day': 18 }  
d12.update({ 'Year': 2019, 'Day': 20, 'Hour': 10 }) # 第一种写法  
# d12.update(Year=2019, Day=20, Hour=10) # 第二种写法  
print(d12)
```

```
{ 'Year': 2019, 'Month': 3, 'Day': 20, 'Hour': 10 }
```

- 没有remove方法，没有delete方法
- 字典方法的使用场景

方法	核心一句话场景	关键优势	代码示例 (假设 d 为字典)
get()	怕报错，想要个默认值	安全访问，避免 <code>KeyError</code>	<code>python
d = {"a": 1}
键不存在时返回 0，不报错
val = d.get("b", 0)
val -> 0
</code>
setdefault()	嵌套结构，自动初始化列表/字典	省去 if key not in 判断	<code>python
d = {}
键不存在则设为空列表，并返回该列表
d.setdefault("tags", []).append("py")
d -> {"tags": ["py"]}
</code>
pop()	用完即焚，取走并删除	原子操作：取值 + 删除	<code>python
d = {"task": "run"}
取出 task 并从字典移除
val = d.pop("task", None)
val -> "run", d -> {}
</code>
update()	配置合并，批量覆盖	快速合并两个字典	<code>python
defaults = {"x": 1, "y": 2}
user = {"y": 99}
defaults.update(user)
defaults -> {"x": 1, "y": 99}
</code>
items()	遍历时既要键又要值	标准迭代模式，解包方便	<code>python
d = {"a": 1, "b": 2}
for k, v in d.items():
 print(f"{k}={v}")
输出: a=1, b=2
</code>
fromkeys()	批量初始化同值字典	快速创建，但小心可变对象陷阱	<code>python
keys = ["x", "y"]
所有键初始化为 0
d = dict.fromkeys(keys, 0)
d -> {"x": 0, "y": 0}
</code>
popitem()	后进先出，移除最新项	实现栈结构 (LIFO)	<code>python
d = {"1": "A", "2": "B"}
移除最后插入的 ("2", "B")
nk, v = d.popitem()
k->"2", v->"B"
</code>

- 字典视图

- keys()、values()、items()方法

返回类似的列表值，分别对应于字典的键、值和键-值对

这些方法返回的值不是真正的列表，它们**不能被修改**，没有append()方法。但这些数据类型可以用于for循环，反映字典的实时变化

```
spam = { 'color': 'red', 'age': 42 }  
for v in spam.values():  
    print(v)  
for v in spam.keys():  
    print(v)  
for v in spam.items():  
    print(v)  
for v in spam.items():  
    print(v)
```

```
red  
42  
color  
age  
( 'color', 'red' )  
( 'age', 42 )  
( 'color', 'red' )  
( 'age', 42 )
```



视图对象：“字典的镜子”，它直接映射底层的字典数据，而不是数据的副本。

1. 动态与实时
这是视图对象最重要的特点。视图对象不存储数据，而是引用原始字典。因此，当原始字典发生变化（增、删、改）时，视图对象会立即同步反映这些变化，无需重新调用方法。
2. 内存高效
由于视图对象只是对原始数据的引用，使用视图对象比转换为列表更节省内存。
3. 支持的操作
行为更像集合（Set）或迭代器
可迭代：可以直接用于 for 循环。
成员检测：支持 in 和 not in 操作，且效率很高（基于哈希查找，平均 O(1)）。
集合运算：键视图（keys）和项视图（items）支持集合运算，如交集（&）、并集（|）、差集（-）等。
不可索引：不支持通过下标访问（如 view 会报错），因为它们不是序列。

循环与迭代

- for循环

```
for iterating_var in sequence:  
    statement1(s)  
else:  
    statement2(s)
```

```
for num in range(10,20): # 迭代 10 到 20 之间的数字
    for i in range(2,num): # 根据因子迭代
        if num%i == 0: # 确定第一个因子
            j=num/i # 计算第二个因子
            print ('%d 等于 %d * %d' % (num, i, j))
            break # 跳出当前循环
        # 循环的 else 部分
    else:
        print (num, '是一个质数')
```

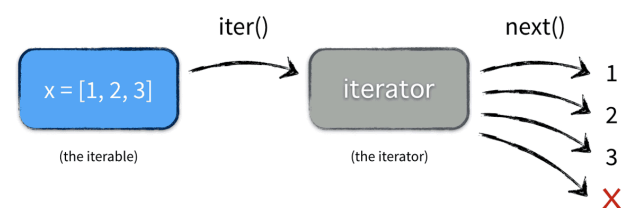
```
10 等于 2 * 5
11 是一个质数
12 等于 2 * 6
13 是一个质数
14 等于 2 * 7
15 等于 3 * 5
16 等于 2 * 8
17 是一个质数
18 等于 2 * 9
19 是一个质数
```

- while循环

```
while expression:
    statement1(s)
statement2(s)
```

```
while expression:
    statement1(s)
else:
    statement2(s)
```

- 迭代



调用可迭代对象的__iter__方法返回一个迭代器对象（iterator）
不断调用迭代器的__next__方法返回元素
知道迭代完成后，处理StopIteration异常

- 可迭代对象

能用for循环进行迭代的对象就是可迭代对象。比如：**字符串，列表，元组，字典，集合，字典视图**等等，都是可迭代对象。

实现了__iter__方法的对象叫做可迭代对象

- 迭代器

实现了 __iter__() 方法：返回自身

实现了 __next__() 方法：负责返回下一个元素，取完就抛出 StopIteration 异常

- zip()、map()、filter()、文件对象，这些返回的都是迭代器。
- 列表、字符串、字典这些，是**可迭代对象**，但不是迭代器。
- **不能直接打印、不能直接查看**，必须用 list() 转成列表才能看到内容
- 迭代器不会一开始就把所有数据加载到内存里，而是等你调用 next() 或者用 for 循环时，才计算并返回下一个值。
- 迭代器的元素被取出来一个，就少一个，**不能回头、不能重复使用**。
- 用迭代器实现循环封装

- zip

zip：把两个按照索引index数组合并成一个新数组，并返回一个数组列表（取决于最短的数组）

```
In [3]: a=('1','2','3','4')
        b=('a','b','c')
        c=zip(a,b)
        next(c)

Out[3]: ('1', 'a')
```

zip数组生成的对象是可迭代对象，并且直接就是一个迭代器

- map

map：传入两个参数，一个是函数，一个是要传入函数的参数所构成的列表
用于将一个列表中每个元素都做一个相同的操作

```
def deal(a):
    return a*a
b=[1,2,3,4]
c=map(deal,b) #mapping 映射 #map 返回值python3为迭代器
for x in c:
    print(x)
```

```
1
4
9
16
```

- filter过滤

和 map() 类似，filter() 也接收一个函数和一个序列。

和 map() 不同的是：filter() 把传入的函数依次作用于每个元素，然后根据返回值是 True 还是 False 决定保留还是丢弃该元素。

```
def f(a):
    return a>0
b=[-1,0,1,2,3]
c=filter(f,b)
for x in c:
    print(x)

1
2
3
```

列表解析

[expr for iter_var in iterable]

- **expr**：每个元素的最终表达式（比如 **x**、**x**2**）
- **iter_var**：循环变量
- **iterable**：可迭代对象（比如 **range(10)**、列表）

a = [x for x in range(10)]

相当于

```
a = []
for i in range(10):
    a.append(i)
```

```
1 a = [] # 定义一个空列表
2
3 def num():
4     for i in range(10): # for循环取出range(10)中的值
5         a.append(i) # 将刚取出的值加入到a中
6         print(a) # 打印a的值
7
8
9 num()
10
```

百家号/干货一条街

```
sqdEvens=[x**2 for x in range(8) if not x%2]
[0,4,16,36]
```

#解析:

```
#for x in range(8): 遍历 0~7
#if not x%2: 筛选条件, 只保留偶数 (x%2 == 0)
#x**2: 对筛选后的偶数做平方运算
```

现代Python风格（Pythonic）中，列表解析通常可以替代 map 和 filter，且可读性更好。

week5

函数

- 函数定义

- 函数是指一段在一起的、可以完成某一功能的程序代码段。
目标：减少代码重复性，提升代码可读性

def 函数名 (参数表) :
函数语句
return 返回值

参数表可选，可以无参，一个或者多个参数，参数类型无限制
返回值可选，可以无返回值，可以返回多个，逗号隔开。实际返回的是元组类型

- pass语句 空函数占位技巧

当暂时不需要实现函数逻辑时，使用pass语句占位，保证程序语法正确，用于快速构建程序框架

```
def func(a,b):
    pass
```

- 参数传递

- 参数传递的本质
 - 传的是对象的引用（地址）
 - 不是纯“值传递”，也不是传统“引用传递”
变量=名字（标签）

Python 传的是“对象的引用”，但变量可以重新绑定，所以不是传统引用传递

↗ C++

```
void func(int &x) {
    x = 100;
}
```

👉 特点是:

- **x** 就是外部变量的“别名”
- 修改 **x** = 修改外部变量本身
- 不能解绑（始终指向同一个变量）

一个人有两个牌

↗ Python

```
def func(x):
    x = 100

a = 10
func(a)
print(a) # 10
```

👉 如果是“引用传递”，这里应该变成 100
👉 但实际上没变

一个人有一个牌，但是自己牌子可以贴在别人身上

C++引用传递

- **x** 是外部变量 **a** 的**别名**，它们完全是同一个东西
- 修改 **x** 就是直接修改 **a** 本身，无法解绑（始终指向同一个变量）
- 就像 “一个人有两个身份证，改哪个都是改本人”

Python参数传递

- 你把 **a** 传给 **func(a)** 时，函数里的 **x** 只是拿到了对象 **10** 的引用（标签）

- 函数里的 `x = 100`，只是把 `x` 这个标签从 `10` 撕下来，贴到了 `100` 上
- 但外面的 `a` 标签，依然牢牢贴在 `10` 上，所以 `a` 不会变
- 就像“给你一个人的名片，你把名片改了，不会影响本人”



参数传递和浅拷贝一样：

- 当操作的是**不可变类型**（数字、字符串、元组）时：修改新变量，原变量**不会变**
- 当操作的是**可变类型**（列表、字典）时：修改对象内部的元素，原变量**会跟着变**

```
def func(x):
    x=x+1
a=10
func(a)
print(a) #10 (没变)
def func(x):
    x.append(4) # 修改的是 x 指向的列表对象本身
a = [1, 2, 3]
func(a)
print(a) # 输出 [1,2,3,4], 变了!
```

参数传递的类型

- 位置参数：fun(a,b,c)
 - 必须传
 - 直接将实参赋予形参，按顺序匹配
- 关键字参数：fun(a=1,b=2,c=3)
 - 根据键值对的形式做实参与形参的匹配
 - 忽略了参数的位置关系，直接根据关键字来进行赋值
 - 参数不必全部赋值，可以使用默认值
- 可变参数
 - fun (*args)
 - 可变位置参数（收集多余的位置参数，*是收集位置参数的标记）
 - 可以传入任意个参数
 - 这些参数都被放到了tuple元组中赋值给行参args，之后要在函数中使用这些行参，直接操作args这个tuple元组

- fun (**kwargs)
 - 可变关键字参数（收集多余的关键字参数）
 - 以键值对字典的形式向函数传参
 - 具有fun(a=1,b=2,c=3)位置的灵活的同时
 - 具有fun (*args)的参数数量上的无限制

参数传递顺序规则：

- 位置参数 → 默认参数 → *args → **kwargs
- 关键字参数必须在位置参数后面

```
def func(a, b=2, *args, **kwargs):
    print(a, b, args, kwargs)

func(1, 3, 4, 5, x=6, y=7)
# 输出: 1 3 (4, 5) {'x': 6, 'y': 7}

"""a → 必传的位置参数
b=2 → 默认参数（可传可不传，不传就用 2）
*args → 收集多余的位置参数，打包成元组
kwargs → 收集多余的关键字参数 **，打包成字典"""
```

```
def func(lst=[]):
    lst.append(1)
    return lst

print(func())
print(func())
print(func())
#输出[1]
[1,1] 第二次调用时，默认列表还是同一个对象!
[1,1,1]
```

原因：默认参数只会在函数定义时创建一次，可变对象的默认参数会被所有调用共享。

```
def func(lst=None):
    if lst is None:
        lst = []
    lst.append(1)
    return lst

用 None 作为默认值，在函数内创建新列表
```

```
1 #P 9
2
3 def test(x, y=5, *a, **b):
4     print(x, y, a, b)
5
6 test(1)
7 test(1, 2)
8 test(1, 2, 3)
9 test(1, 2, 3, 4)
10 test(x=1)
11 test(x=1, y=1)
12 test(x=1, y=1, a=1)
13 test(1, y=1)
14 test(1, 2, 3, 4, a=1)
15 test(1, 2, 3, 4, k=1, t=2, o=3)
16 test(1, 2, y=1)
```

```
1 5 0 {}
2 0 {}
3 (3,) {}
4 (3, 4) {}
5 0 {}
6 1 0 {}
7 1 0 {'a': 1}
8 1 0 {}
9 (3, 4) {'a': 1}
10 (3, 4) {'k': 1, 't': 2, 'o': 3}

TypeError                                Traceback (most recent call last)
<ipython-input-12-5917d9c860e1> in <module>
     14 test(1, 2, 3, 4, a=1)
     15 test(1, 2, 3, 4, k=1, t=2, o=3)
--> 16 test(1, 2, y=1)

TypeError: test() got multiple values for argument 'y'
```

参数传递的本质

共享传递

- 共享传参指函数的各个形参获得实参中各个引用的副本。也就是说，函数内部的形参是实参的别名
- 形参和实参一开始指向同一个对象，但形参可以重新绑定到新对象上，不会影响实参
 - 对于绝大多数情况下，在函数内部直接修改形参的值不会影响实参，而是创建一个新变量。
 - 在有些情况下，可以通过特殊的方式在函数内部修改实参的值。
 - 如果传递给函数的实参是可变序列，并且在函数内部使用下标或可变序列自身的原地操作方法增加、删除元素或修改元素时，实参也得到相应的修改。



- 不可变对象（数字、字符串、元组）：函数内修改不影响外部因为不可变对象不能被原地修改，函数内的修改本质上是**给形参重新绑定新对象**，外部实参的标签还贴在原对象上，所以不会变。
- 可变对象（列表、字典）：函数内原地修改会影响外部因为可变对象支持**原地修改**，函数内通过下标 / 方法修改对象本身时，形参和实参指向的是同一个对象，所以外部也会看到变化。

Lambda函数

lambda 参数列表: 表达式

- lambda x, y: x*y
函数输入是x和y，输出是它们的积xy
- lambda:None
函数没有输入参数，输出是None
- lambda *args: sum(args)
输入是任意个数的参数，输出是它们的和(隐性要求是输入参数必须能够进行加法运算)

- lambda **kwargs: 1
输入是任意键值对参数，输出是1

```
nums = [1, 2, 3]
res = map(lambda x: x * 2, nums)
print(list(res)) # [2, 4, 6]
```

[2, 4, 6]

```
nums = [1, 2, 3, 4]
res = filter(lambda x: x % 2 == 0, nums)
print(list(res)) # [2, 4]
```

[2, 4]

```
data = [(1, 3), (2, 1), (4, 2)]
data.sort(key=lambda x: x[1])
print(data)
```

[(2, 1), (4, 2), (1, 3)]

```
data = ["apple", "banana", "pear"]
print(max(data, key=lambda x: len(x)))
```

banana

递归函数

- 定义：在函数内调用当前函数本身的函数就是递归函数
- 必要条件：要有明确的终止条件，否则程序会无限递归直至崩溃
- 经典案例：斐波那契数列计算、阶乘、二叉树遍历、二分查找算法
- Python默认递归深度限制为1000，过深需手动调整

模块

模块定义

是一个 Python 文件，以 .py 结尾，包含了 Python 对象定义和Python语句

- 能够有逻辑地组织 Python 代码段。
- 把相关的代码分配到一个模块里能让你的代码更好用，更易懂。
- 模块能定义函数，类和变量，模块里也能包含可执行的代码。

模块导入

- 一个 .py 文件就是一个模块
- 通过 import 导入，可复用代码
- import math
from math import sqrt
from math import * # 不推荐
import math as m

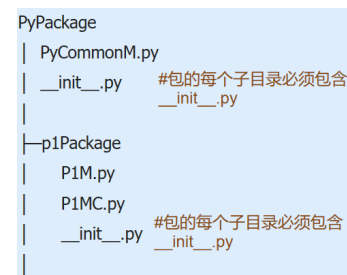


导入一个模块，Python 解析器对模块位置的搜索顺序是：

- 1、当前目录
 - 2、如果不在当前目录，Python 则搜索在 shell 变量 PYTHONPATH 下的每个目录。
set PYTHONPATH=c:\python27\lib;
 - 3、如果都找不到，Python会察看默认路径。
- 模块搜索路径存储在 system 模块的 sys.path 变量中。变量里包含当前目录，PYTHONPATH和由安装过程决定的默认目录。

模块包

- 大型系统中，会根据代码功能将模块文件放在多个目录中。存放模块文件被称为目录包
- 包和子包的每个目录，都必须包含 `__init__.py`



三种导入方式

- 导入包根目录下的模块
`from PyPackage import PyCommonM`
- 导入子包中的模块
`from PyPackage.p1Package import P1M`
- 导入子包中更深 的模块
`from PyPackage.p1Package.P1MC import P1MC`

`__init__.py` 的作用

- 用于标识目录是一个 Python 包，并可包含包的初始化代码（可以写代码，在包第一次被导入时执行）或导入子模块（`__init__.py` 就是普通 Python 文件，能写变量、函数、类、import，没有任何限制）
- 可以为空文件

变量作用域

变量作用域（LEGB规则）

变量的作用域决定了在哪一部分程序可以访问哪个特定的变量名称。

- L（Local）局部作用域：
定义函数内部的变量
- E（Enclosing）闭包函数外的函数中：
嵌套作用域（enclosing 作用域，外层非全局作用域）
- G（Global）全局作用域：
定义在函数外的变量
- B（Built-in）内建作用域：
系统内固定模块里定义的变量，如预定义在builtin 模块内的变量

系统以 **L → E → G → B** 的规则查找，即：在局部找不到，便会去局部外的局部找（例如闭包），再找不到就会去全局找，再者去内建中找

闭包（Closure）

当一个函数内部引用了外部作用域的变量，并且这个函数被返回或长期使用，就形成闭包

```

def outer():
    x = 10
    def inner():
        return x
    return inner

f = outer()
print(f()) # 10

```

inner保留了x

inner 用到了外部变量 x

outer 已经执行完了

但 x 没消失！

正常情况下：函数结束后，x 应该被销毁

但闭包中：Python 会把变量“封存”起来 你可以理解为：函数把外部变量“带走了”

```

def outer():
    x = 10
    def inner():
        return x
    x = 20
    return inner

f = outer()
print(f())

```

20

```

def outer():
    x = 10
    return lambda: x

f = outer()
print(f()) # 10

```

10

```

def make_adder(n):
    return lambda x: x + n

add5 = make_adder(5)
print(add5(10)) # 15

```

15

Lambda 是函数，本质上也可以形成闭包

修改作用域外的变量

- 模块（module），类（class）以及函数（def、lambda）能引入新的作用域
- 代码块（如 if/elif/else/、try/except、for/while等）是不能引入新的作用域的，也就是说这些语句内定义的变量，外部也可以访问

global

当内部作用域想修改外部作用域的变量时，就要用到 **global** 进行声明

```
a = 10
def test():
    a = a + 1
    print(a)
```

```
test()
```

#会报错

错误原因: UnboundLocalError

Python 在编译函数时, 只要看到函数内有给变量 a 赋值的操作, 就会默认把 a 当成局部变量。

当执行 `a = a + 1` 时, 它会先把右边的 a 当成局部变量, 但此时 a 还没有被赋值, 所以会报 “局部变量未绑定” 的错误。

即使外面有全局变量 `a=10`, Python 也不会去读取它, 因为函数里的赋值操作已经把 a 标记为局部变量了。

```
a = 10
def test():
    global a
    a = a + 1
    print(a)
```

```
test() # 输出 11
```

关键操作: `global a`

这行代码的作用是告诉 Python: “我要在函数里使用 / 修改的 a 是全局变量, 不是局部变量!”

执行 `a = a + 1` 时, Python 会去读取全局作用域里的 `a=10`, 计算后把 a 修改为 11

此时 `print(a)` 输出的就是修改后的全局变量值 11

如果函数里出现了赋值操作, Python 会直接把这个变量标记为局部变量, 不再去外层 / 全局找。

而 `global` 关键字可以打破这个规则, 强制让函数里的变量指向全局作用域。

如果只是读取全局变量, 不需要 `global`

只有当你需要修改全局变量时, 才需要用 `global` 声明

■ nonlocal

如果要修改嵌套作用域 (当一个函数里面又定义了另一个函数时, 就形成了嵌套结构) (enclosing 作用域, 外层非全局作用域) 中的变量则需要 `nonlocal` 关键字

`nonlocal` 关键字, 就是专门用来 修改外层非全局作用域 (也就是外层函数里的变量) 的。

```
def outer():
    num = 10
    def inner():
        num = 100 # 这里没有nonlocal
        print(num)
    inner()
    print(num)
```

```
outer()
```

输出:

100

10

为什么会这样?

1. 内层函数 `inner()` 里写了 `num = 100`, Python 会默认把它当成局部变量。

2. 这相当于在内层创建了一个新的局部变量 num, 和外层的 `num=10` 完全没关系。

3. 所以内层 `print(num)` 输出的是 100, 外层 `print(num)` 输出的还是原来的 10。

```
def outer():
    num = 10
    def inner():
        nonlocal num # 关键声明
        num = 100
        print(num)
    inner()
    print(num)
```

```
outer()
```

输出:

100

100

关键: `nonlocal num` 做了什么?

它告诉 Python: “我在内层函数里要修改的 num, 不是我自己的局部变量, 而是外层函数 `outer()` 里的那个 num!”

1. 内层执行 `num = 100` 时, 会直接修改外层函数里的 num, 而不是创建新变量。

2. 所以内层 `print(num)` 输出 100, 外层 `print(num)` 也会输出被修改后的 100。

异常处理

• 异常

异常即是一个事件，该事件会在程序执行过程中发生，影响了程序的正常执行。

一般情况下，在Python无法正常处理程序时就会发生一个异常。

异常是Python对象，表示一个错误。

当Python脚本发生异常时我们需要捕获处理它，否则程序会终止执行。

◦ 常见异常

- AssertionError：当assert断言条件为假的时候抛出的异常
- AttributeError：当访问的对象属性不存在的时候抛出的异常
- IndexError：超出对象索引的范围时抛出的异常
- KeyError：在字典中查找一个不存在的key抛出的异常
- NameError：访问一个不存在的变量时抛出的异常
- OSError：操作系统产生的异常
- SyntaxError：语法错误时会抛出此异常
- TypeError：类型错误，通常是不通类型之间的操作会出现此异常
- ZeroDivisionError：进行数学运算时除数为0时会出现此异常

• 异常捕捉和处理

```
try:
    <statements>           #运行try语句块，并试图捕获异常
except <name1>:
    <statements>           #如果name1异常发现，那么执行该语句块。
except (name2, name3):
    <statements>           #如果元组内的任意异常发生，那么捕获它
except <name4> as <variable>:
    <statements>           #如果name4异常发生，那么进入该语句块，并把异常实例命名为variable
except:
    <statements>           #发生了以上所有列出的异常之外的异常
else:
    <statements>           #如果没有异常发生，那么执行该语句块
finally:
    <statement>            #无论是否有异常发生，均会执行该语句块。
```

◦ try...except...else

检测try语句块中的错误，从而让except语句捕获异常信息并处理

```
try:
    <语句>           #运行别的代码
except <名字>:
    <语句>           #如果在try部份引发了'name'异常
except <名字>, <数据>:
    <语句>           #如果引发了'name'异常，获得附加的数据
else:
    <语句>           #如果没有异常发生
```

```
try:
    stu='jack'
    print(stu)
except BaseException as msg:
    print(msg) #捕获系统弹出的异常信息
else:
    print("stu is defined!")

1 try:
2     fh = open("testfile", "w")
3     fh.write("这是一个测试文件，用于测试异常!!")
4 except IOError:
5     print("Error: 没有找到文件或读取文件失败")
6 else:
7     print("内容写入文件成功")
8     fh.close()
9
10
11
```

Error: 没有找到文件或读取文件失败

◦ try-finally

语句无论是否发生异常都将执行最后的代码

```
try:
    <语句>
finally:
    <语句> #退出try时总会执行
raise
```

```
1 try:
2     fh = open("testfile", "w")
3     fh.write("这是一个测试文件，用于测试异常!!")
4 finally:
5     print ("Error: 没有找到文件或读取文件失败")

Error: 没有找到文件或读取文件失败
```

• 抛出异常

◦ raise语句用来手动抛出一个异常

- raise #可以在raise语句之前创建该实例或者在raise语句中创建。
- raise #Python会隐式地创建类的实例
- raise name(value) #抛出异常的同时，提供额外信息value
- raise # 把最近一次产生的异常重新抛出来
- raise exception from E

```
try:
    1/0
except Exception as E:
    raise TypeError('bad input') from E
```

执行的结果如下：

```
Traceback (most recent call last):
  File "hh.py", line 2, in <module>
    1/0
ZeroDivisionError: division by zero

The above exception was the direct cause of the following exception:

Traceback (most recent call last):
  File "hh.py", line 4, in <module>
    raise TypeError('bad input') from E
TypeError: bad input
```

```
def division(x,y):
    if y==0:
        raise ZeroDivisionError("Zero is not allow!")
    return x/y

try:
    division(8,0)
except BaseException as msg:
    print(msg)
```

- Assert 断言
 - Assert 语句是将调试断言插入程序的一种方便的方法
 - assert 条件,报错信息（可选） #如果条件不成立，直接抛出AssertionError错误，程序中断
- ```
age = 15
assert age >= 18, "年龄必须大于等于18岁"
print('已成年')
#输出: AssertionError: 年龄必须大于等于18岁
```
- 特点：`assert` 抛出的 `AssertionError` 默认不带任何描述信息，除非你在断言语句中显式提供了错误信息
    - `traceback` 模块
      - 作用：查看异常的传播轨迹，追踪异常触发的源头（比如具体是哪个文件、哪一行代码出了问题）。
      - 常用方法：`traceback.print_exc()` 可以打印完整的异常栈信息，比普通的 `print(e)` 更详细。

## week6

### 传统文件操作

- 为什么需要文件？
  - 程序中的数据存在内存中，关机丢失
  - 需要持久化存储：保存到硬盘文件
  - 应用场景：保存用户设置、日志、处理数据文件、读取配置文件
- 文件操作三步骤
  - 打开、读写、关闭文件
- 传统文件操作基础
  - 打开文件：`open()`
    - 语法：`f = open(文件路径, 模式)`

- 核心模式：

| 模式  | 说明                |
|-----|-------------------|
| 'r' | 只读（默认），文件必须存在     |
| 'w' | 写入，不存在则创建，存在则覆盖   |
| 'a' | 追加，不存在则创建，不覆盖原有内容 |
| 'x' | 独占创建，文件已存在则报错     |



当需要创建一个新文件并写入内容，但如果文件已存在则不应覆盖原有内容→用x

- 关闭文件：`close()`
  - 语法：`f.close()`
  - 警告：忘记关闭会导致数据丢失、资源泄露
- 文件读取方法
  - `.read()`：一次性读取整个文件为字符串，大文件占内存
  - `.readline()`：逐行读取（含换行符），配合循环使用
  - `.readlines()`：读取所有行，返回字符串列表
  - 迭代器读取：直接 for 循环遍历文件对象（推荐）

```
全量读取
with open('poem.txt','r',encoding='utf-8') as f:
 whole = f.read()

逐行读取
with open('data.txt','r',encoding='utf-8') as f:
 line = f.readline()
 while line:
 print(line.strip())
 line = f.readline()

读取所有行
with open('data.txt','r',encoding='utf-8') as f:
 lines = f.readlines()

迭代器读取
with open('data.txt','r',encoding='utf-8') as f:
 for line in f:
 print(line.strip())
```

- 文件写入方法

- `.write()`: 写入单个字符串, 需手动加 `\n` 换行
- `.writelines()`: 写入多个字符串, 不自动换行

```
覆盖写入
with open('output.txt', 'w', encoding='utf-8') as f:
 f.write("第一行\n")
追加写入
with open('output.txt', 'a', encoding='utf-8') as f:
 f.write("追加行\n")
多字符串写入
lines = ["apple\n", "banana\n"]
with open('fruits.txt', 'w') as f:
 f.writelines(lines)
```

- 上下文管理器with
  - 优势: **自动关闭文件**, 即使发生异常也能执行
  - 语法: `with open(...) as 变量:`
  - 强烈推荐, 不需要写`close()`, 替代传统 `open()+close()` 离开缩进会自动关闭
- 编码问题
  - 不同操作系统默认编码不同 (Windows 常用 gbk, Linux/Mac 常用 utf-8)
  - 建议总是显式指定 `encoding='utf-8'`
  - 常见错误: `UnicodeDecodeError` → 编码不匹配
- 文件路径
  - **相对路径**: `data/info.txt`、`../output.txt` (相对于执行命令的目录)
  - **绝对路径**: `C:/xxx/file.txt`、`/home/xxx/file.txt` (固定路径)
    - 查看当前工作目录:  
`import os`  
`print(os.getcwd())`
    - 关键注意: 如果程序在 `/home/user/script/run.py`, 但是你在 `/home/user/` 下执行 `python script/run.py` 此时 `os.getcwd()` 返回 `/home/user`, 而不是 `/home/user/script`。  
相对路径**相对于执行命令的目录**, 而非.py 文件所在目录
  - 路径使用场景:

| 场景 | 推荐 | 原因 |
|----|----|----|
|----|----|----|

|                                          |      |             |
|------------------------------------------|------|-------------|
| 脚本内读取同目录下的配置文件                           | 相对路径 | 便于整体移动项目文件夹 |
| 读取系统固定位置的文件 (如 <code>/etc/hosts</code> ) | 绝对路径 | 位置绝对不变      |
| 跨计算机共享代码                                 | 相对路径 | 避免写死个人电脑的路径 |
| 临时调试                                     | 绝对路径 | 简单快速        |

```
with open('hello.txt', 'w', encoding='utf-8') as f:
 f.write("Hello, Python!\n")

with open('hello.txt', 'r', encoding='utf-8') as f:
 print(f.read())

with open('hello.txt', 'a', encoding='utf-8') as f: #a追加模式下无法执行read(), 必须再次with open
 f.write("Learning file I/O\n")

with open('hello.txt', 'r', encoding='utf-8') as f:
 print(f.read())
```

Pathlib: 面向对象的路径操作 `from pathlib import Path`

- 核心优势
  - 传统方式用字符串拼接路径: 容易出错, 跨平台麻烦
  - `pathlib` 提供 `Path` 对象, 用 `/` 拼接, 自动适配操作系统
  - Python 3.4+ 内置, 无需安装
- 基础用法
  - 创建 `Path` 对象: `Path('projects')`  
创建了一个表示路径的`Path`对象, 这个路径是当前目录下的一个名为`projects`的目录
  - 路径拼接: `Path('projects') / 'src' / 'main.py'`  
使用 `/` 操作符来拼接路径
  - 核心属性

| 属性                  | 作用                              |
|---------------------|---------------------------------|
| <code>name</code>   | 文件名 (含后缀): <code>main.py</code> |
| <code>stem</code>   | 文件名主干 (无后缀): <code>main</code>  |
| <code>suffix</code> | 文件后缀: <code>.py</code>          |
| <code>parent</code> | 父目录: <code>projects/src</code>  |

## 💡 比字符串路径更安全、更可读! 📌

### ◦ 常用方法

#### 1. 检查类:

- `p.exists()`: 路径是否存在
- `p.is_file()`: 是否为文件
- `p.is_dir()`: 是否为目录

#### 2. 创建目录:

- `Path('data/images').mkdir(parents=True, exist_ok=True)`
  - `Path('data/images')`: 创建一个 Path 对象, 表示路径 data/images, 指向当前工作目录下的 data子目录中的 images子目录
  - `.mkdir()`: 创建目录, 如果目录已存在, 默认会抛出 FileExistsError异常
  - `parents=True`: 如果为 True, 会自动创建路径中所有不存在的父目录。例如, 如果 data目录不存在, 会先创建 data, 再在 data下创建 images
  - `exist_ok=True`: 如果为 True, 当目标目录已存在时不会引发异常, 静默继续。如果为 False (默认值), 目录已存在时会抛出 FileExistsError

### • 用path简化读写文件

#### ◦ 读取: `p.read_text()`

参数:

encoding: 编码格式

errors: 编码错误处理方式

#### ◦ 写入: `p.write_text()`

参数:

data: 要写入的字符串

encoding: 编码格式, 默认使用系统编码, 建议指定 'utf-8'

errors: 编码错误处理方式

返回值: 返回写入的字符数

### • 遍历与通配符

- `glob('*.txt')`
  - 在当前目录 (不包含子目录) 中匹配所有扩展名为 .txt的文件
  - 不会递归搜索子目录

```
列出当前目录下所有 .txt 文件
for file in Path('.').glob('*.txt'):
 print(file)
```

#### ◦ `rglob('*.txt')`

- 在当前目录及其所有子目录中递归匹配所有扩展名为 .txt的文件
- 相当于 `glob('**/*.txt')`

```
递归遍历所有 .py 文件
for file in Path('.').rglob('*.py'):
 print(file)
```

### • 重命名与删除

#### ◦ 重命名: `src.rename(dst)`

- 删除文件: `dst.unlink(missing_ok=True)` (不存在不报错)  
它相当于 `os.remove()` 或 `os.unlink()` 函数, 但以面向对象的方式使用。

### • 临时文件: tempfile 模块

#### ◦ 临时文件的需求

- 程序中间结果不需要长期保存
- 不想手动管理文件名和清理
- 例如: 下载临时缓存、测试数据

#### ◦ 匿名临时文件

- 无文件路径, 仅进程内访问, 退出自动删除

```
import tempfile
with tempfile.TemporaryFile(mode='w+t') as f:
 f.write('临时数据')
 f.seek(0) #将文件指针重置到开头
 print(f.read())
```

#### ◦ 临时目录

- 创建系统唯一临时目录, 退出自动删除所有内容

```
with tempfile.TemporaryDirectory() as tmpdir:
 print('临时目录:', tmpdir)
```

```
p = Path(tmpdir) / 'data.txt'
p.write_text('内容')
```

💡 根据 Python tempfile 模块的行为，TemporaryDirectory 通常作为上下文管理器（with 语句）使用，**当退出 with 语句块时，临时目录会被自动删除并清理**。如果不使用 with，则在对象被垃圾回收时也会清理，但题目中强调“自动清理”的典型场景是退出 with 语句块。

- 有名字的临时文件
  - 系统自动生成唯一文件名，支持指定后缀
  - `delete=True`：关闭文件自动删除（默认）

```
with tempfile.NamedTemporaryFile(suffix='.txt', delete=True) as tmp:
 print('临时路径:', tmp.name)
 tmp.write(b'hello')
```

## CSV JSON

### • CSV 文件操作

#### ◦ 基础定义

CSV = 逗号分隔值，纯文本表格格式，每行一条记录

- 行内字段用逗号，分隔（也可以用其他分隔符，如制表符 \t）
- 第一行通常是列名（header）
- 可以用 Excel、记事本打开

#### ◦ 读取 CSV

- **列表形式：** `csv.reader()`
  - 每行返回列表，需记住列索引
  - `next(reader)` 获取表头
- **字典形式：** `csv.DictReader()`
  - 首行自动作为键，`row['列名']` 读取，可读性高

```
name,age,city
Alice,28,New York
Bob,32,London
```

```
列表读取
import csv
with open('people.csv', newline='', encoding='utf-8') as f:
 reader = csv.reader(f)
 header = next(reader) # 第一行: ['name', 'age', 'city']
 for row in reader:
 print(row['name']) # ['Alice', '28', 'New York']
 print(row[0], row[1]) # Alice 28
```

#### # 字典读取

#代码可读性高: `row['name']` 比 `row[0]` 清晰

#不依赖列顺序

```
import csv
with open('people.csv', newline='', encoding='utf-8') as f:
 reader = csv.DictReader(f)
 for row in reader:
 print(row['name'], row['age']) # ['Alice', '28']
```

`newline=''` 是必须的（避免空行问题）

`reader` 返回的每一行是字符串列表

需要自己记住第几列是什么含义（容易错）

### ◦ 写入 CSV

1. **列表数据：** `csv.writer()` + `writerows()`
2. **字典数据：** `csv.DictWriter()` + `writeheader()`（写表头）

#### # 列表写入

```
data = [['name', 'age'], ['Alice', 28]]
with open('output.csv', 'w', newline='', encoding='utf-8') as f:
 writer = csv.writer(f)
 writer.writerows(data) # 一次性写入多行
 # writer.writerow(row) # 逐行写入
```

#### # 字典写入

```
data = [{'name': 'Alice', 'age': 28}]
with open('output.csv', 'w', newline='', encoding='utf-8') as f:
 writer = csv.DictWriter(f, fieldnames=['name', 'age'])
 writer.writeheader() # 写入表头
 writer.writerows(data)
```



- 使用csv.writer写入时，所有数据会被自动转换为字符串
- writerow( )方法接收一个列表参数，写入一行数据
- 写入CSV时必须使用newline=""参数避免空行
- csv.DictWriter 在写入数据前并非必须调用 writeheader() —— 表头可以手动写入或省略

#### 常见问题与解决方案

| 常见问题    | 表现                    | 解决方案                                |
|---------|-----------------------|-------------------------------------|
| 字段内包含逗号 | "New York, NY" 会被拆成两列 | 字段用引号包裹: "New York, NY", csv 模块自动处理 |
| 字段内包含换行 | 行结构被破坏                | 同样用引号包裹, csv 模块支持                   |
| 编码错误    | UnicodeDecodeError    | 指定 encoding='utf-8'                 |
| 空行      | 写入后每两行之间有一个空行         | 使用 newline="" 参数                    |

#### Json

##### JSON 文件操作

- 基础定义
- 轻量级数据交换格式，易于人阅读和编写，也易于机器解析
- 基于 JavaScript 语法，但已成为语言无关的标准
- 支持的数据类型：  
字符串、数字、布尔（true/false）、null  
数组 [ ... ]（对应 Python 列表）  
对象 { ... }（对应 Python 字典）

##### 语法规则

- 整个文件只能有一个根值（可以是对象、数组或字面量）
- 对象中的键（key）必须用双引号包裹
- 字符串值必须用双引号，支持转义：\"、\\、\n、\t、\uXXXX（Unicode）
- 最后一个成员后面不能有逗号（与 Python 不同）
- 不能有注释（部分解析器支持，但不标准）
- 数字不能有前导零（如 0123 非法），但可以是 0.5 或 -10

```
{
 "name": "Alice",
 "age": 28,
 "skills": ["Python", "Data Analysis"],
 "is_student": false
}
```

```
{ name: "Alice" } // 错误：键没有双引号
{ "name": 'Alice' } // 错误：值用了单引号
{ "a": 1, } // 错误：多余逗号
```

```
{
 "name": "Alice",
 "age": 28,
 "address": {
 "city": "New York",
 "zip": 10001
 },
 "hobbies": ["reading", "coding"]
}
```

#### json模块的转换表

| Python     | JSON   |
|------------|--------|
| dict       | object |
| list/tuple | array  |
| str        | string |
| int/float  | number |
| True       | true   |
| False      | false  |
| None       | null   |

#### 核心方法

##### 1. 文件操作：

- 读取： `json.load(文件对象)`

```
import json
with open('data.json', 'r', encoding='utf-8') as f:
 data = json.load(f) # 直接得到 Python 对象 (dict 或 list)
 print(data['name']) # Alice
 print(data['skills'][0]) # Python
```



JSON 中的 true / false / null 会变成 Python 的 True / False / None  
键必须是双引号，字符串也必须是双引号（不能用单引号）

- 写入：`json.dump(数据, 文件对象, indent=2, ensure_ascii=False)`

```
data = {
 "name": "Alice",
 "age": 28,
 "skills": ["Python", "Data Analysis"]
}

with open('output.json', 'w', encoding='utf-8') as f:
 json.dump(data, f, indent=2, ensure_ascii=False)
```



indent=2: 美化输出, 缩进 2 空格 (不写则所有内容挤在一行)  
ensure\_ascii=False: 允许输出中文 (否则会变成 \uXXXX)

## 2. 字符串与Json互转 (内存操作)

- 转 JSON 字符串: `json.dumps(数据)`
- 转 Python 对象: `json.loads(JSON字符串)`

```
import json
Python 对象 → JSON 字符串
person = {"name": "Bob", "age": 32}
json_str = json.dumps(person) # '{"name": "Bob", "age": 32}'
JSON 字符串 → Python 对象
obj = json.loads(json_str) # {'name': 'Bob', 'age': 32}
```

### • 嵌套结构读取

- {
  - "school": "第一中学",
  - "grade": 3,
  - "classes": [
    - {
      - "class\_name": "三班",
      - "students": [
        - {"name": "张三", "score": 92},
        - {"name": "李四", "score": 88}

```
]
},
{
 "class_name": "四班",
 "students": [
 {"name": "王五", "score": 95}
]
}
]
```

- 示例: `data["classes"][0]["students"][0]["name"]`

```
name,math,english
Alice,90,85
Bob,78,92
Charlie,88,84
Diana,95,90
Eve,85,88
Frank,92,87
Grace,79,91
Henry,88,82
Iris,91,89
Jack,76,94
```

```
import csv
import json
```

```
存储结果的列表
totals = []
```

```
1. 用 csv.DictReader 读取 CSV 文件
with open("scores.csv", newline="", encoding="utf-8") as csvfile:
 reader = csv.DictReader(csvfile)
 for row in reader:
 # 计算总分 (注意转成int)
 math = int(row["math"])
 english = int(row["english"])
 total = math + english
 # 存入列表
 totals.append({
 "name": row["name"],
```

```

 "total": total
 })

2. 保存为 totals.json
with open("totals.json", "w", encoding="utf-8") as f:
 json.dump(totals, f, ensure_ascii=False, indent=2)

print("已生成 totals.json 文件")

import json

读取 totals.json
with open("totals.json", "r", encoding="utf-8") as f:
 data = json.load(f)

找出总分最高的人
max_student = max(data, key=lambda x: x["total"])

print(f"总分最高的人是: {max_student['name']}, 总分: {max_student['total']}")

```

## week7

### Tkinter 编程基础

- GUI核心要素：窗口 → 组件 → 布局 → 事件循环



1. **Tkinter 定位**：内置到 Python 安装包，安装 Python 即可 import；IDLE 由 Tkinter 编写

#### 2. Tkinter 编程步骤

- 导入 Tkinter 模块
- 创建控件
- 指定控件的 master（归属父容器）
- 告知布局管理器（geometry manager）控件已创建

3. **Tkinter 组件清单**：Label、Button、Entry、Text、CheckBox、RadioButton、Listbox、Scale、Spinbox、Menu、Combobox、Frame

4. **Tkinter 布局分类**：绝对布局、相对布局、表格布局

5. 数据显示

- 表格数据
- 树状数据

```

import tkinter as tk
创建主窗口，容纳整个GUI程序
root = tk.Tk()
设置窗口标题
root.title("NEW")
添加Label组件，显示文本
theLabel= tk.Label(root,text="就是体验一下")
调用pack()自动调节尺寸
theLabel.pack()
窗口显示，启动事件循环
root.mainloop()

```

```

import tkinter as tk
import os
root = tk.Tk()
仅允许一个根窗口，其他用Toplevel
w= tk.Label(root)
创建PhotoImage对象（支持gif、pgm、ppm）
bm = tk.PhotoImage(file='C:\课程\pytho 12026\17\lclas \Austria.gif')
保留引用，防止被垃圾回收
w.image= bm
设置显示图片
w['image']=bm
w.pack()
okButton = tk.Button(root,text="确定")
设置背景色
okButton['background']='yellow'
等价写法：okButton.configure(background='yellow')
okButton.pack()
root.mainloop()

```

- Tkinter 常用组件

| Tkinter 类名  | 简介                      |
|-------------|-------------------------|
| Toplevel    | 顶层容器，为组件提供单独容器，类似窗口     |
| Button      | 按钮组件，响应用户点击             |
| Canvas      | 画布，绘制直线、矩形、椭圆、多边形、位图等   |
| Checkbutton | 复选框，可供用户勾选              |
| Entry       | 单行输入框，用户可输入单行内容         |
| Frame       | 容器，装载其他 GUI 组件          |
| Label       | 标签，显示不可编辑的文本或图标         |
| LabelFrame  | 带标题的容器，类似 Frame         |
| Listbox     | 列表框，列出多个选项供选择           |
| Menu        | 菜单组件                    |
| Menubutton  | 菜单按钮，包含下拉 / 层叠式菜单       |
| OptionMenu  | Menubutton 子类，菜单按钮      |
| Message     | 消息框，显示多行文本，现被 Label 替代  |
| PanedWindow | 分区窗口，可拖动分隔线调整区域         |
| Radiobutton | 单选钮，组内仅一个可选中            |
| Scale       | 滑动条，拖动设定数值              |
| Spinbox     | 微调选择器，上下箭头选值            |
| Scrollbar   | 滚动条，为文本域 / 画布 / 列表框提供滚动 |
| Text        | 多行文本框，显示 / 编辑多行文本       |

tkinder组件修改属性

组件实例对象["属性名"] = 值  
entry['state'] = 'readonly'

标签 Label

用于在窗口中显示文本或图片。它通常用于提示信息、标题、状态说明，用户无法直接编辑其内容。

```
import tkinter as tk
root = tk.Tk()
label1 = tk.Label(root, text="Hello, Tkinter!")
label2 = tk.Label(root, bitmap="error")
label1.pack()
label2.pack()
root.mainloop()
```

| 属性           | 作用                  | 示例值                                       |
|--------------|---------------------|-------------------------------------------|
| text         | 显示文字                | "用户名："                                    |
| font         | 字体（家族，大小，样式）        | ("微软雅黑",16,"bold")                        |
| fg/bg        | 前景色 / 背景色           | "red","#FFCC00"                           |
| bitmap       | 显示内置 10 种标准图标       | "error","info","warning"                  |
| image        | 显示图片（PhotoImage 对象） | photo                                     |
| width/height | 宽度 / 高度（字符数 / 像素）   | 20,100                                    |
| justify      | 多行文本对齐              | "left","center","right"                   |
| wraplength   | 文本换行宽度（像素）          | 150                                       |
| anchor       | 文本在标签中的位置           | "n","s","e","w","center"                  |
| relief       | 边框样式                | "flat","raised","sunken","groove","ridge" |
| padx/pady    | 标签内边距（水平 / 垂直）      | 5,10                                      |



Python 中image属性仅支持gif、pgm、ppm格式，bitmap属性支持xbm格式，如需更多格式可安装Pillow并使用ImageTk.PhotoImage。

```
photo = PhotoImage(file="image.gif")
bmp = BitmapImage(file="logo.xbm")
label = Label(root,image = photo)
```

注意：Python内置了10种位图，可以直接使用，设置bitmap即可。“error”“gray75”“gray50”“gray25”“gray12”“hourglass”“info”“questhead”“question”“warning”



输入框 Entry → 单行输入框

用于接收单行文本输入的组件，类似于网页中的文本框。它常用于用户名、密码、搜索关键词等简短信息的录入。

```
import tkinter as tk
root = tk.Tk()
entry = tk.Entry(root)
entry.pack()
root.mainloop()
```

| 属性    | 作用        | 示例值          |
|-------|-----------|--------------|
| width | 显示宽度（字符数） | 20           |
| font  | 字体        | ("Arial",12) |

| 属性           | 作用                              | 示例值                                                                                                                         |
|--------------|---------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| fg/bg        | 前景色 / 背景色                       | "black","white"                                                                                                             |
| show         | 隐藏输入字符（密码）                      | "*" 或 "."                                                                                                                   |
| state        | 状态：<br>normal/readonly/disabled | "normal"默认状态，用户可以正常输入、修改内容。<br>"readonly" 状态下，用户无法修改内容，只读状态，但可以通过 <b>textvariable.set()</b> 改变显示值。<br>"disabled" 状态下完全不可交互。 |
| justify      | 文本对齐                            | "left","center","right"                                                                                                     |
| textvariable | 绑定 StringVar，双向同步               | var                                                                                                                         |

• StringVar 方法

是 Tkinter 的特殊变量类型，可以实现变量与 Entry 内容的双向自动同步

| 方法                   | 作用                       |
|----------------------|--------------------------|
| set(value)           | 设置变量值，Entry 同步显示         |
| get()                | 获取变量当前值                  |
| trace(mode,callback) | 监听变量变化（w 写入 /r 读取 /u 删除） |



注意：Entry没有Text属性！  
采用StringVar类型进行输入值跟踪，以保证值的变更随时可以显示在界面上。

```
from tkinter import *
root = Tk()
e = StringVar()
使用textvariable属性，绑定字符串变量e
entry1 = Entry(root, textvariable = e)
e.set('请输入.....')
entry1.pack()

entry2 = Entry(root, show="*")
entry2.pack()

root.mainloop()
```

• 变量监听代码

```
import tkinter as tk
def on_change(*args):
 print(f"变量被修改了！当前值 = {var.get()}")
 print(f"回调参数: {args}")
root = tk.Tk()
var = tk.StringVar()
var.trace("w", on_change)
entry = tk.Entry(root, textvariable=var)
```

```
entry.pack()
root.mainloop()
```

命令按钮 Button

交互组件，用于响应用户的点击操作。用户点击按钮时，会触发预先绑定的函数或方法，从而执行特定任务（如提交表单、打开文件、计算等）

```
b = Button(master, option=value, ...)
```

master: 代表承载该按钮的父容器

options: 可选项，即该按钮的可设置的属性。这些选项可以用键 = 值的形式设置，并以逗号分隔

```
import tkinter as tk
def say_hello():
 print("Hello, Tkinter!")
root = tk.Tk()
btn = tk.Button(root, text="点我", command=say_hello)
btn.pack()
root.mainloop()
```

| 属性           | 作用                        | 示例值                |
|--------------|---------------------------|--------------------|
| text         | 按钮文本                      | "确定"               |
| command      | 点击调用函数（不加括号）              | my_func            |
| font         | 字体                        | ("微软雅黑",12,"bold") |
| fg/bg        | 前景色 / 背景色                 | "white","blue"     |
| width/height | 宽度 / 高度（字符数 / 像素）         | 10,2               |
| state        | 状态：normal/disabled/active | "disabled"         |
| relief       | 边框样式                      | "raised","sunken"  |
| image        | 显示图片（PhotoImage）          | photo              |
| compound     | 图片与文字位置                   | "left","right"     |
| cursor       | 鼠标悬停光标                    | "hand2","cross"    |

• 传参方法（原文）：使用 lambda 或 functools.partial

```
lambda传参
btn = tk.Button(root, text="传参", command=lambda: my_func("参数1", "参数2"))
partial传参
from functools import partial
```

```
btn = tk.Button(root, text="传参", command=partial(my_func, "参数1", "参数2"))

#编写一个程序，实现两个输入框内容的交换

from tkinter import *

def change():
 temp=e1.get()
 e1.set(e2.get())
 e2.set(temp)

root = Tk()
root.title("tk")

e1=StringVar()
entry1 = Entry(root, textvariable=e1)
entry1.pack()

e2=StringVar()
entry2=Entry(root, textvariable=e2)
entry2.pack()

button1=Button(root, text="交换", command=change)
button1.pack()

root.mainloop()
```

```
'''问候生成器设计一个简单的 GUI 程序，包含以下组件：
一个 Label，显示提示文字：“请输入你的名字：”
一个 Entry，用于用户输入姓名
一个 Button，上面写着“打招呼”
另一个 Label，用于显示问候语（初始为空）
功能要求：用户在 Entry 中输入姓名，点击“打招呼”按钮后，下方的 Label 显示 你好，[姓名]！。
如果用户没有输入任何内容（即 Entry 为空）就点击按钮，则显示 请先输入姓名。
（可选）增加一个“清空”按钮，点击后清空 Entry 和问候语 Label 的内容。
'''

from tkinter import *
```

```
def hi():
 name=entry1.get()
 label2['text']=f"你好，{name}! "

def clear():
 entry1.delete(0, END) #不能用entry1['text']=''
 label2['text']='' #label2.config(text="")

root= Tk()

label1=Label(root, text="请输入你的名字： ")
label1.pack()

entry1=Entry(root)
entry1.pack()

button1=Button(root, text="打招呼", command=hi)
button1.pack()

button2=Button(root, text="清空", command=clear)
button2.pack()
```

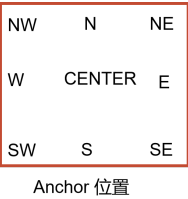
Tkinter 布局

指控制窗体容器中各个控件（组件）的位置关系  
布局方式有三种: 绝对布局(place), 相对布局(pack)和网格布局(grid)

- 绝对布局 place  
窗口的变化对位置没有影响

| 属性     | 取值                         | 简析                  | 含义                                                                                                                             |
|--------|----------------------------|---------------------|--------------------------------------------------------------------------------------------------------------------------------|
| anchor | N/E/S/W/NW/NE/SW/SE/CENTER | 组件在容器中的位置，默认 CENTER | 1. 控制组件在 place 分配的空间中的位置<br>2. "n", "ne", "e", "se", "s", "sw", "w", "nw", 或 "center" 来定位（ewsn代表东西南北，上北下南左西右东）<br>3. 默认值是 "nw" |
| x/y    | 整数                         | 组件左上角绝对坐标（像素）       | 1. 指定该组件的水平/垂直偏移位置（像素）                                                                                                         |

| 属性                 | 取值             | 简析               | 含义                                                   |
|--------------------|----------------|------------------|------------------------------------------------------|
|                    |                |                  | 2. 如同时指定了 relx 选项, 优先实现 relx 选项                      |
| relx/rely          | 0~1 浮点数        | 相对父容器位置, 0.5 为正中 | 1. 指定该组件相对于父组件的水平/垂直位置<br>2. 取值范围 0.0 ~ 1.0          |
| width/height       | 非负整数           | 组件宽高 (像素)        | 指定该组件的宽度/高度 (像素)                                     |
| relwidth/relheight | 0~1 浮点数        | 相对父容器的宽高         | 1. 指定该组件相对于父组件的宽度/高度<br>2. 取值范围 0.0 ~ 1.0            |
| bordermode         | INSIDE/OUTSIDE | 边框模式, 默认 INSIDE  | 1. 指定边框模式 ("inside" 或 "outside")<br>2. 默认值是 "inside" |
| in_                |                |                  | 1. 将该组件放到该选项指定的组件中<br>2. 指定的组件必须是该组件的父组件             |



- 计算规则: 优先计算 relx/rely, 再实现 x/y 偏移  
relx 和 rely 选项指定的是相对于父组件的位置, 范围是 0.0 ~ 1.0, 因此 0.5 表示位于正中间。  
relwidth 和 relheight 选项则是指定相对于父组件的尺寸

```
import tkinter as tk
root = tk.Tk()
tk.Label(root, bg="red").place(relx=0.5, rely=0.5, relheight=0.75, relwidth=0.75, anchor="center")
tk.Label(root, bg="yellow").place(relx=0.5, rely=0.5, relheight=0.5, relwidth=0.5, anchor="center")
tk.Label(root, bg="green").place(relx=0.5, rely=0.5, relheight
```

```
=0.25, relwidth=0.25, anchor="center")
root.mainloop()
```

- place类提供了下列函数 (使用组件实例对象调用) (拓展)
  - place\_slaves(): 以列表方式返回本组件的所有子组件对象
  - place\_configure(option=value): 给place布局管理器设置属性, 使用属性 option=value方式设置
  - propagate(boolean): 设置为True表示父组件的几何大小由子组件决定(默认值), 反之则无关
  - place\_info(): 返回place提供的选项所对应得值
  - grid\_forget(): Unpack组件, 将组件隐藏并且忽略原有设置, 对象依旧存在, 可以用pack(option, ...), 将其显示
  - location(x, y): x/y为以像素为单位的点, 函数返回此点是否在单元格中, 在哪个单元格中。返回单元格行列坐标, (-1, -1)表示不在其中
  - size(): 返回组件所包含的单元格, 揭示组件大小

相对布局 pack

组件位置或大小的变化会随着窗口的变化而变化

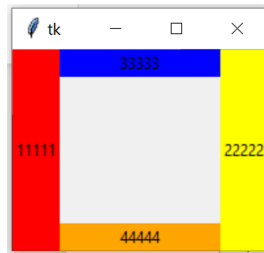
| 属性名         | 属性简析                               | 取值                                      | 取值说明                                    |
|-------------|------------------------------------|-----------------------------------------|-----------------------------------------|
| fill        | 设置组件是否向水平或垂直方向填充                   | X、Y、BOTH、NONE                           | fill=X (水平方向填充), BOTH (水平和垂直), NONE 不填充 |
| expand      | 设置组件是否展开                           | YES、NO (1、0)                            | expand=YES                              |
| side        | 设置组件的对齐方式                          | LEFT、TOP、RIGHT、BOTTOM                   | 值为左、上、右、下                               |
| ipadx/ipady | 设置x方向 (或者y方向) 内部间隙 (子组件之间的间隔)      | 可设置数值, 默认是0                             | 非负整数, 单位为像素                             |
| padx/pady   | 设置x方向 (或者y方向) 外部间隙 (与之并列的组件之间的间隔)  | 可设置数值, 默认是0                             | 非负整数, 单位为像素                             |
| anchor      | 锚选项, 当可用空间大于所需求的尺寸时, 决定组件被放置于容器的何处 | N、E、S、W、NW、NE、SW、SE、CENTER (默认值为CENTER) | 表示八个方向以及中心                              |

Expand=1展开时有效

组件排列方式, 默认 (TOP) 一个组件独占一行, 从上往下排

```
from tkinter import *
window = Tk()
label1=Label(window, text='11111', bg='red')
label2=Label(window, text="22222", bg='yellow')
label3=Label(window, text='33333', bg='blue')
label4=Label(window, text='44444', bg='orange')
label1.pack(side=LEFT, fill=Y)
```

```
label2.pack(side=RIGHT, fill=Y)
label3.pack(side=TOP, fill=X)
label4.pack(side=BOTTOM, fill=X)
window.mainloop()
```



- pack类提供了下列函数（使用组件实例对象调用）（拓展）
  - pack\_slaves(): 以列表方式返回本组件的所有子组件对象
  - pack\_configure(option=value): 给pack布局管理器设置属性，使用属性option=value方式设置
  - propagate(boolean): 设置为 True 表示父组件的几何大小由子组件决定（默认值），反之则无关。
  - pack\_info(): 返回pack提供的选项所对应得值。
  - pack\_forget(): Unpack组件，将组件隐藏并且忽略原有设置，对象依旧存在，可以用pack(option, ...), 将其显示。
  - location(x, y): x, y为以像素为单位的点，函数返回此点是否在单元格中，在哪个单元格中。返回单元格行列坐标，(-1, -1)表示不在其中
  - size(): 返回组件所包含的单元格，揭示组件大小
- 网格布局 grid（最推荐）

程序大多数都是矩形的界面，可以把它划分为几行几列的网格，然后根据行号和列号，将组件放置于网格之中

使用grid布局时，需要在里面指定两个参数，分别用row 表示行，column表示列

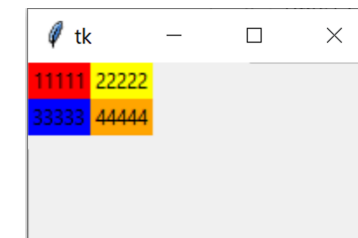
💡 需要注意的是row和column的序号都从0开始

表格中的每一行的高度，以这一行最高组件为基准。  
表格中的第一列的宽度，以这一列最宽组件为基准。



| 属性名         | 属性简述                              | 取值                                 | 取值说明                   |
|-------------|-----------------------------------|------------------------------------|------------------------|
| row/column  | row 为行号，column 为列号，设置将组件放置于第几行第几列 | 取值为行、列的序号，不是行数与列数                  | row 和 column 的序号都从0 开始 |
| sticky      | 设置组件在网格中的对齐方式                     | N, E, S, W, NW, NE, SW, SE, CENTER | 类似于 pack 布局中的锚选项       |
| rowspan     | 组件所跨越的行数                          | 跨越的行数                              | 取值为跨越占用的行数，而不是序号       |
| columnspan  | 组件所跨越的列数                          | 跨越的列数                              | 取值为跨越占用的列数，而不是序号       |
| ipadx/ipady | 设置x方向（或者y方向）内部间隙（子组件之间的间隔）        | 可设置数值，默认是0                         | 非负整数，单位为像素             |
| padx/pady   | 设置x方向（或者y方向）外部间隙（与之并列的组件之间的间隔）    | 可设置数值，默认是0                         | 非负整数，单位为像素             |

```
from tkinter import *
window = Tk()
label1=Label(window,text='11111',bg='red')
label2=Label(window,text='22222',bg='yellow')
label3=Label(window,text='33333',bg='blue')
label4=Label(window,text='44444',bg='orange')
label1.grid(row=0,column=0)
label2.grid(row=0,column=1)
label3.grid(row=1,column=0)
label4.grid(row=1,column=1)
window.mainloop()
```



```
1 from tkinter import *
2 window = Tk()
3 window.config(bg="blue")
4 window.title("Grid")
5
6 for i in range(29):
7 for j in range(29):
8 if (i+j)%2==1:
9 Label(window,text=str(i+j)+"AA",bg="pink").grid(row=i, column=j)
10 else:
11 Label(window,text=str(i+j),bg="green").grid(row=i, column=j)
12
13 window.mainloop()
```



Tkinter布局

控制变量，值与组件对象的文本/值关联  
BooleanVar IntVar DoubleVar StringVar

```
from tkinter import *
master = Tk()
var = IntVar()

Label(master, text="First").grid(sticky=E)
Label(master, text="Second").grid(sticky=E)

e1 = Entry(master)
e2 = Entry(master)

e1.grid(row=0, column=1)
e2.grid(row=1, column=1)

checkboxbutton = Checkbutton(master, text='Preserve aspect', variable=var)
checkboxbutton.grid(columnspan=2, sticky=W)

photo = PhotoImage(file='D:\\课程\\python\\2019\\5\\\\baubinia-s.gif')

label = Label(image=photo)
label.image = photo
label.grid(row=0, column=2, columnspan=2, rowspan=2, sticky=W+E+N+S, padx=5, pady=5)

button1 = Button(master, text='Zoom in')
button1.grid(row=2, column=2)

button2 = Button(master, text='Zoom out')
button2.grid(row=2, column=3)

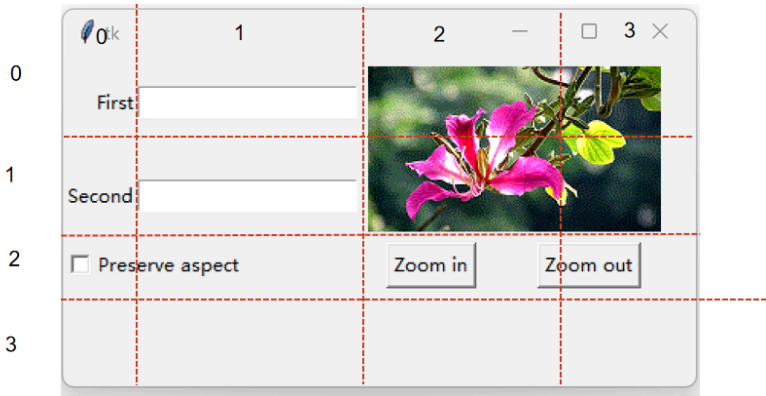
mainloop()
```

没有为label控件指定具体的位置，column从0开始，row从第一个没有使用的值开始  
Sticky=E，与网格的东/右侧对齐

Columnspan=2  
占据第0和1列，2列

尺寸规则

- 行高：该行最高组件高度 + 内边距
- 列宽：该列最宽组件宽度 + 内边距



Label1 [row=0, column=0, sticky=E]  
Label2 [row=1, column=0, sticky=E]  
Entry1 [row=0, column=1]  
Entry1 [row=1, column=1]  
checkboxbutton [row=2, column=0, columnspan=2, sticky=W]  
labelG [row=0, column=2, columnspan=2, rowspan=2]  
sticky=W+E+N+S, padx=5, pady=5]  
button1 [row=2, column=2]  
button2 [row=2, column=3]

Frame框架

在屏幕上创建一块矩形区域，多作为容器来布局窗体，或进行控件分组

bd:指定边框宽度  
Relief:指定边框样式，SUNKEN为凹陷

```
from tkinter import *
root=Tk()
frame1=Frame(bd=2,relief=SUNKEN)
frame2=Frame(bd=2,relief=SUNKEN)

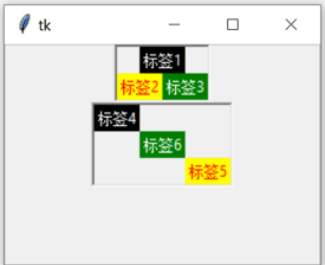
label1=Label(frame1,text="标签1",fg='white',bg='black')
label2=Label(frame1,text="标签2",fg='red',bg='yellow')
label3=Label(frame1,text="标签3",fg='white',bg='green')
label4=Label(frame2,text="标签4",fg='white',bg='black')
label5=Label(frame2,text="标签5",fg='red',bg='yellow')
label6=Label(frame2,text="标签6",fg='white',bg='green')

frame1.pack()
frame2.pack()

label1.pack()
label2.pack(side=LEFT)
label3.pack(side=RIGHT)

label4.grid(row=1,column=1)
label5.grid(row=3,column=4)
label6.grid(row=2,column=2)

root.mainloop()
```



布局对比（原文）

| 布局方式      | 一句话理解         | 适用场景               | 核心方法                             |
|-----------|---------------|--------------------|----------------------------------|
| place绝对布局 | 像贴贴纸，精确指定像素位置 | 需要完全控制坐标（如画图工具、游戏） | place(x, y)                      |
| pack相对布局  | 像堆箱子，按顺序依次排列  | 简单垂直/水平工具栏、状态栏     | pack(side=TOP/BOTTOM/LEFT/RIGHT) |
| grid网格布局  | 像填表格，按行、列对齐   | 大多数表单、对话框（最推荐）     | grid(row, column)                |

鼠标和键盘事件

一个Tkinter应用生命周期中的大部分时间都处在一个消息循环中。它等待事件的发生: 事件可能是按键按下, 鼠标点击, 鼠标移动等。

Tkinter提供了用以处理相关事件的机制, 处理函数可以被绑定给各个控件的各种事件

如果相关事件发生, 回调函数会被触发, 事件对象event会传递给回调函数

在 Tkinter 中, 事件是指用户与界面交互时发生的动作, 例如鼠标点击、按键按下、窗口移动等。通过 事件绑定 (bind 方法), 可以让程序在特定事件发生时自动执行指定的函数, 实现交互式应用

通过bind 方法绑定事件: widget.bind("<事件描述符>", 回调函数)

- widget: 要绑定事件的组件 (如 Label、Button、Canvas 等)
- <事件描述符>: 字符串, 表示事件类型 (用尖括号包围)
- 回调函数: 事件发生时要调用的函数, 该函数必须接收一个 事件对象 event 作为参数
- Tkinter事件处理机制的事件冒泡原则

- 事件先被 触发它的子组件 捕获并处理
- 处理完后，事件会像气泡一样向上冒，传递给它的父组件
- 继续向上传递给顶层窗口（root）
- 直到所有父级都处理完，或主动阻止冒泡

鼠标事件全表

| Event             | Description |                   |             |
|-------------------|-------------|-------------------|-------------|
| <Triple-Button-3> | 鼠标右键三击      |                   |             |
| <Button-1>        | 鼠标左键        | <Triple-Button-2> | 鼠标中键三击      |
| <Button-3>        | 鼠标右键        | <B1-Motion>       | 左键移动        |
| <Button-2>        | 鼠标中键        | <B3-Motion>       | 右键移动        |
| <Button-4>        | 鼠标向上滚动      | <B2-Motion>       | 中键移动        |
| <Button-5>        | 鼠标向下滚动      | <ButtonRelease-1> | 释放鼠标左键      |
| <Double-Button-1> | 鼠标左键双击      | <ButtonRelease-3> | 释放鼠标右键      |
| <Double-Button-3> | 鼠标右键双击      | <ButtonRelease-2> | 释放鼠标中键      |
| <Double-Button-2> | 鼠标中键双击      | <Enter>           | 鼠标光标进入控件时触发 |
| <Triple-Button-1> | 鼠标左键三击      | <Leave>           | 鼠标光标离开控件时触发 |

键盘事件全表

| 键盘事件             | 描述                     |                 |                          |
|------------------|------------------------|-----------------|--------------------------|
| <BackSpace>      | 退格                     |                 |                          |
| <Key>            | 响应所有的按键(按下)            | <a>/<b>         | 指定的小写字母键                 |
| <KeyRelease>     | 响应所有的按键(松开)            | <A>/<Z>         | 指定的大写字母键                 |
| <FocusIn>        | 控件或控件的子空间获得键盘焦点.       | <Control-Alt-a> | 组合键(可识别任意组合键)            |
| <FocusOut>       | 控件丢失键盘焦点 (焦点移动到另一个控件). | char            | 从键盘输入的和按键事件的相关字符         |
| <Return>/<Enter> | 只有回车键响应                | keycode         | 从按键输入的和按键事件的键代码(ASCII码)  |
| <Escape>         | esc键                   | keysym          | 从按键输入的和按键事件的键符号(即字符)     |
| <space>          | 空格键                    | num             | 按键数字(1, 2, 3)表明按下的是哪个鼠标键 |
| <Tab>            | Tab键                   | widget          | 触发这个事件的小构件对象             |

|                            |                                |               |                        |
|----------------------------|--------------------------------|---------------|------------------------|
| <Up>/<Down>/<Left>/<Right> | 上下左右键                          | x和y           | 当前鼠标在小构件中以像素为单位的位置     |
| <Shift_L>/<Shift_R>        | 左右Shift键(类似有左右两个键的, 添加_L/_R区分) | x_root和y_root | 当前鼠标相对于屏幕左上角的以像素为单位的位置 |

event 对象属性全表

| 属性                         | 含义                            |
|----------------------------|-------------------------------|
| event.x, event.y           | 鼠标相对于组件的坐标（像素）                |
| event.x_root, event.y_root | 鼠标相对于屏幕左上角的坐标                 |
| event.keysym               | 按键的符号名（如 "a", "Return", "Up"） |
| event.keycode              | 按键的数字码                        |
| event.widget               | 触发事件的组件对象                     |
| event.char                 | 按键对应的字符（仅适用于可打印字符）            |

```
from tkinter import *
def onleftClick(event):
 label1.config(text="单击了鼠标左键")
def onrightClick(event):
 label1.config(text="单击了鼠标右键")
def onleftDrag(event):
 label1.config(text="按下鼠标拖动, 位置(%s,%s)"%(event.x,event.y))
def onReturn(event):
 label1.config(text="按下了Return键")
def onkeyPress(event):
 label1.config(text="按下了%s键"%event.char)
def onArrowPress(event):
 label1.config(text='按下了上箭头')
label1=Label(text='你好,python')
label1.pack()
bt1=Button(text='按钮')
bt1.bind("<Button-1>",onleftClick)
bt1.bind("<Button-3>",onrightClick)
bt1.bind("<B1-Motion>",onleftDrag)
bt1.bind("<Return>",onReturn)
bt1.bind("<KeyPress>",onkeyPress)
bt1.bind("<Up>",onArrowPress)
bt1.pack()
bt1.focus()
```



```
#实现一个按钮，绑定鼠标进入时改变背景色，离开时恢复原色（使用 <Enter> 和 <Leave>）

from tkinter import *

def change(event):
 event.widget["bg"]="blue"

def recover(event):
 event.widget["bg"]="SystemButtonFace"

root=Tk()

b1=Button(root,text="未知按钮")
b1.bind("<Enter>",change)
b1.bind("<Leave>",recover)
b1.pack()

root.mainloop()
```

用户名与密码的校验 📌

```
label1=Label(fup,text='用户名:',width=8,anchor=E)
label1.grid(row=1,column=1)
entry1=Entry(fup,textvariable=username,width=20) #用户名输入组件
docheck1=entry1.register(usercheck)
entry1.config(validate='all',validatecommand=(docheck1,'%P'))
entry1.grid(row=1,column=2)

label2=Label(fup,text='密码:',width=8,anchor=E)
label2.grid(row=2,column=1)
entry2=Entry(fup,show='*',textvariable=password,width=20) #密码输入组件
docheck2=entry2.register(passwordcheck)
entry2.config(validate='all',validatecommand=(docheck2,'%d','%S'))
entry2.grid(row=2,column=2)

fdown=Frame()
fdown.pack()
bt1=Button(fdown,text='重置',command=reset)
bt1.grid(row=1,column=1)

bt2=Button(fdown,text='确定',command=done)
bt2.grid(row=1,column=2)

label3=Label()
label3.pack()
mainloop()
```

## 用户名与密码的校验

用户密码

```
def usercheck(what):
 if len(what)>10:
 label3.config(text='用户名不能超过10个字符',fg='red')
 return False
 return True

def passwordcheck(why,what):
 if why=='1':
 if what not in '0123456789':
 label3.config(text='密码只能是数字',fg='red')
 return False
 return True

def reset(): #重置按钮命令函数
 entry1.delete(0,END)
 password.set('')
 label3.config(text='')

def done(): #确定按钮命令函数
 if username.get()=="DM" and password.get()=="5818":
 label3.config(text='欢迎回来! ')
 else:
 label3.config(text='你输入的用户名: %s, 密码为: %s 不正确!'%(username.get(),password.get()))
```

## 用户名与密码的校验

### 1. 需求（原文）

- 用户名≤10 个字符
- 密码只能是数字

- 用户名 DM、密码 5818 → 显示 “欢迎回来”，否则提示不正确

## 2. 文本校验核心（原文）

- validatecommand: 绑定验证函数，仅返回 True/False
- 验证参数（原文）：
  - % P: 输入框最新文本内容
  - % d: 动作代码（0 删除 / 1 插入 / 2 焦点 / 变量修改）
  - % S: 插入 / 删除的文本
- validate 选项（原文）：
  - focus: 获得 / 失去焦点验证
  - focusin: 获得焦点验证
  - focusout: 失去焦点验证
  - key: 编辑时验证
  - all: 所有情况验证
  - none: 关闭验证（字符串 "none"）

## 3. 组件代码

```
用户名校验
docheck1=entry1.register(usercheck)
entry1.config(validate='all',validatecommand=(docheck1,'%
P'))
密码校验
docheck2=entry2.register(passwordcheck)
entry2.config(validate='all',validatecommand=(docheck2,'%
d,%s'))
```

## 4. 函数代码（原文）

```
def usercheck(what):
 if len(what)>10:
 label3.config(text='用户名不能超过10个字符',fg='red')
 return False
 return True
def passwordcheck(why, what):
 if why=='1':
 if what not in '0123456789':
 label3.config(text='密码只能是数字',fg='red')
 return False
```

```
return True
def reset():
 entry1.delete(0, END)
 password.set("")
 label3.config(text='')
def done():
 if username.get()=="DM" and password.get()=="5818":
 label3.config(text='欢迎回来!')
 else:
 label3.config(text='用户名:%s,密码不正确!'%username.ge
t())
```

# week8

## 画布 Canvas

是一个多功能的组件，用于绘制图形、显示图像、创建动画或实现自定义控件。它支持各种形状（线条、矩形、椭圆、多边形等）、文本、图像，并可以响应用户事件（鼠标、键盘）。

### • 创建代码（原文）

```
import tkinter as tk
root = tk.Tk()
canvas = tk.Canvas(root, width=400, height=300, bg='white')
canvas.pack()
tk.mainloop()
```

### 1. 绘制方法（原文）

- create\_arc (coord,start,extent,fill): 绘制圆弧

```
coord = 10,50,240,210
arc = canvas.create_arc(coord,start=0,extent=150,fill="blue")
```

- create\_image (x,y,anchor,image): 创建图像

```
filename = PhotoImage(file = "sunshine.gif")
image = create_image(50,50,anchor=NE,image=filename)
```

- create\_line (x0,y0,x1,y1,...): 创建线条

```
line = canvas.create_line(x0,y0,x1,y1,...,xn,yn,options)
```

- create\_oval (x0,y0,x1,y1): 创建圆 / 椭圆

```
oval = canvas.create_oval(x0,y0,x1,y1,options)
```

- create\_polygon (顶点坐标): 创建至少有三个顶点的多边形

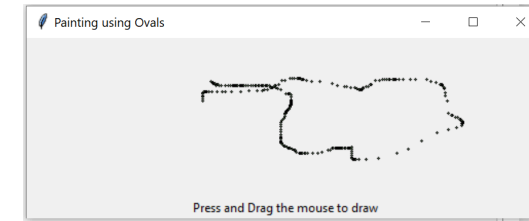
```
polygon = canvas.create_polygon(x0,y0,x1,y1,...,xn,yn,options)
```

- create\_rectangle (x0,y0,x1,y1): 创建矩形

```
rectangle = canvas.create_rectangle(x1,y1,x2,y2)
```

#随手画

```
from tkinter import *
canvas_width=500
canvas_height=150
def paint(event):
 python_green = "#476042"
 x1, y1 = (event.x - 1), (event.y - 1) #event.x、event.y:
 鼠标在画布上的当前位置
 x2, y2 = (event.x + 1), (event.y + 1) #x1,y1 和 x2,y2: 定义了一个 2x2 像素的正方形区域
 w.create_oval(x1, y1, x2, y2, fill=python_green)
master = Tk()
master.title("Painting")
w= Canvas(master, width=canvas_width, height=canvas_height)
w.pack(expand=YES,fill=BOTH) #组件把左右上下都填满,必须配合expand=True才会真正拉伸
w.bind("<B1-Motion>",paint)
message = Label(master,text="按住鼠标拖动画图")
message.pack(side=BOTTOM)
mainloop()
```



```
import math
from tkinter import *

window=Tk()
screenx = 400
screeny = 400
canvas = Canvas(window,width = screenx,height = screeny,bg = 'white')

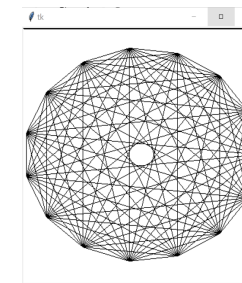
AspectRatio = 0.85
MAXPTS = 15
h = screeny
w = screenx
xcenter = w / 2
ycenter = h / 2
radius = (h - 30) / (AspectRatio * 2) - 20
step = 360 / MAXPTS
angle = 0.0

points=[]
for i in range(MAXPTS):
 angle = step*i
 rads = angle * math.pi / 180.0
 px = xcenter+int(math.cos(rads) * radius)
 py = ycenter-int(math.sin(rads) * radius * AspectRatio)
 points.append((px,py))

canvas.create_oval(xcenter-radius,ycenter-radius,xcenter+radius,ycenter-radius)

for i in range(MAXPTS):
 for j in range(i,MAXPTS):
 canvas.create_line(points[i][0],points[i][1],points[j][0],points[j][1])

canvas.pack()
mainloop()
```



## 单选按钮 Radiobutton

即在同一组内只能有一个按钮被选中，每当选组内的一个按钮时，其它的按钮自动改为非选中态

- 分组规则（原文）：绑定相同变量为同一组

```

from tkinter import *
def sel():
 selection = "You selected option"+str(var.get())
 print(selection)

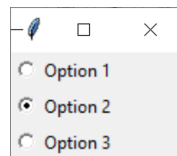
root = Tk()
#创建整型变量，用于绑定，相同的整型变量视为同一组
var= IntVar()
R1 = Radiobutton(root, text="Option1", variable=var, value=1, command=sel)
R1.pack(anchor=W)

R2=Radiobutton(root, text="Option2", variable=var, value=2, command=sel)
R2.pack(anchor=W)

R3=Radiobutton(root, text="Option3", variable=var, value=3, command=sel)
R3.pack(anchor=W)

root.mainloop()

```



字体设置小练习（原文）：通过 Radiobutton 选择颜色、字体、大小，实时修改 Label 样式

## 复选框 Checkbutton

选取我们需要的选项，它前面有个小正方形的方块，如果选中则有一个对号，也可以再次点击以取消该对号来取消选中。

```

from tkinter import *
def sel():
 print(str(CheckVar1.get()), str(CheckVar2.get()))

top = Tk()

CheckVar1=IntVar()
CheckVar2=IntVar()

```

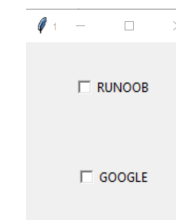
```

CheckVar1.set(1)
CheckVar2.set(0)

C1=Checkbutton(top, text="RUNOOB", variable=CheckVar1, \
 onvalue=1, offvalue=0, command=sel)
C2=Checkbutton(top, text="GOOGLE", variable=CheckVar2, \
 onvalue=1, offvalue=0, command=sel)

C1.pack()
C2.pack()
top.mainloop()

```



## 微调框 Spinbox

Entry小窗口部件的变体，它可以用来选择一个固定的值  
当用户只需要从极少的数值中进行选择的时候，就可以使用Spinbox取代Entry

```

from tkinter import *

def changefont():
 labell.config(font=(fontname.get(), fontsize.get()))

root=Tk()
labell=Label(text='大家好!')
labell.pack(expand=1, fill=X)

fontname=StringVar()
fontname.set('隶书')
fontsize=IntVar()
fontsize.set(15)
changefont()

spin1=Spinbox(textvariable=fontname, values=("隶书", "楷体", "宋体"), \
 wrap=True, command=changefont) #wrap=True选择项是否
以形成循环
spin1.pack()
spin2=Spinbox(textvariable=fontsize, from_=15, to=25, \

```

```

 increment=1,wrap=True,command=changeFont)
spin2.pack()
root.mainloop()

```

## 列表框 Listbox

显示一组文本选项，以列表的形式来提供选项的（后两个是通过按钮的形式）

### 核心方法

- activate(index)  
-- 将给定索引号对应的选项激活（在其文本下方画一条下划线）
- curselection()  
-- 返回一个元组，包含被选中的选项的序号（从 0 开始）-- 如果没有选中任何选项，返回一个空元组
- delete(first, last=None)  
-- 删除参数 first 到 last 范围内（包含 first 和 last）的所有选项-- 如果忽略 last 参数，表示删除 first 参数指定的选项
- get(first, last=None)  
-- 返回一个元组，包含参数 first 到 last 范围内（包含 first 和 last）的所有选项的文本-- 如果忽略 last 参数，表示返回 first 参数指定的选项的文本
- index(index)  
-- 返回与 index 参数相应的选项的序号（例如 lb.index("end")）
- insert(index, \*elements)  
-- 添加一个或多个项目到 Listbox 中-- 使用 lb.insert("end") 添加新选项到末尾
- itemcget(index, option)  
-- 获得 index 参数指定的项目对应的选项（由 option 参数指定）

```

import tkinter as tk
master = tk.Tk()

theLB = tk.Listbox(master)
for item in ["鸡蛋", "鸭蛋", "鹅蛋", "李狗蛋"]:
 theLB.insert("end", item)
theLB.grid(columnspan=2)

e=tk.StringVar()
entry1=tk.Entry(master, textvariable=e)
button1=tk.Button(master, text="Add", \
 command=lambda x=theLB:x.insert("end",e.get()))
button2= tk.Button(master, text="Delete", \

```

```

command=lambda x=theLB:x.delete("active"))

```

```

entry1.grid(row=1,column=2)
button1.grid(row=1,column=0)
button2.grid(row=1,column=1)
master.mainloop()

```



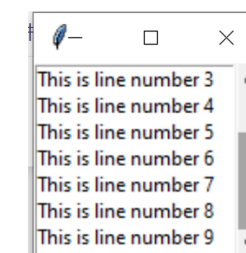
## 滚动条 Scrollbar

可以单独使用，但最多的还是与其它控件（Listbox,Text,Canvas等）结合使用

```

from tkinter import *
#初始化Tk()
myWindow = Tk()
#设置标题
myWindow.title("Scrollbar")
scrollbar = Scrollbar(myWindow)
scrollbar.pack(side=RIGHT, fill=Y)
mylist = Listbox(myWindow, yscrollcommand=scrollbar.set)
for line in range(10):
 mylist.insert(END,"This is line "+str(line))
 mylist.pack(side=LEFT,fill=BOTH)
scrollbar.config(command=mylist.yview)
myWindow.mainloop()

```



## 文本组件 Text

用于显示和处理多行文本

在Tkinter的所有组件中，Text组件显得异常强大和灵活，它适用于处理多任务，虽然该组件的主要目的是显示多行文本，但它常常被用于作为简单的文本编辑器和网页浏览器使用。

```
#创建文本
root = Tk()
text = Text(root)
```

#### • 核心方法

##### ◦ 插入文本内容

text.insert(index,string) index = x.y 的形式,x表示行, y表示列  
向第一行插入数据,text.insert(1.0,'hello world')

##### ◦ 清空python Tkinter Text控件文本的方法

#从第一行清除到最后一行  
text.delete(1.0,Tkinter.END)

简单记事本

```
def docopy():
 data=text1.get(SEL_FIRST, SEL_LAST) #获得选中内容
 text1.clipboard_clear() #清除剪贴板
 text1.clipboard_append(data) #将内容写入剪贴板

def docut():
 data=text1.get(SEL_FIRST, SEL_LAST)
 text1.delete(SEL_FIRST, SEL_LAST) #删除选中内容
 text1.clipboard_clear()
 text1.clipboard_append(data)

def dopaste():
 text1.insert(INSERT, text1.clipboard_get()) #插入剪贴板内容

def doclear():
 text1.delete('1.0', END) #删除全部内容

def doopen(): #打开文件, 将文件内容读到Text组件中
 filename=askopenfilename() #选择要打开的文件
 if filename=='':
 return 0 #没有选择要打开的文件, 直接返回
 filestr=open(filename, 'rb').read() #获得文件内容
 text1.delete('1.0', END)
 text1.insert('1.0', filestr.decode('utf-8')) #将文件内容按UTF-8格式解码后写入文本框
 text1.focus()

def dosave(): #将Text组件内容写入文件
 filename=asksaveasfilename() #获得写入文件的名字
 if filename:
 data=text1.get('1.0', END) #获得文本框内容
 open(filename, 'w').write(data) #写入文件
```

```
frame1=LabelFrame(relief=GROOVE, text=' 工具栏: ')
frame1.pack(anchor=NW, fill=X)
bt1=Button(frame1, text=' 复制')
bt1.grid(row=1, column=1)
bt2=Button(frame1, text=' 剪切')
bt2.grid(row=1, column=2)
bt3=Button(frame1, text=' 粘贴')
bt3.grid(row=1, column=3)
bt4=Button(frame1, text=' 清空')
bt4.grid(row=1, column=4)
bt5=Button(frame1, text=' 打开...')
bt5.grid(row=1, column=5)
bt6=Button(frame1, text=' 保存...')
bt6.grid(row=1, column=6)

sc=Scrollbar()
sc.pack(side=RIGHT, fill=Y)
text1=Text()
text1.pack(expand=YES, fill=BOTH)
text1.config(yscrollcommand=sc.set)

bt1.config(command=docopy)
bt2.config(command=docut)
bt3.config(command=dopaste)
bt4.config(command=doclear)
bt5.config(command=doopen)
bt6.config(command=dosave)
sc.config(command=text1.yview) #将滚动条关联到文本框内置垂直滚动条方法
mainloop()
```

## 文件选择框 filedialog

让用户直观地先择一个或者多个文件或者保存文件等操作

#### • 核心函数

- \*\*askopenfilename(): 打开一个文件
- \*\*askopenfilenames(): 打开一组文件
- \*\*asksaveasfilename(): 保存文件

```
sfname = filedialog.askopenfilename(title='选择Excel文件', filetypes=[('Excel', '*.xlsx'), ('All Files', '*')])
```

initialdir: 指定初始打开的目录。

initialfile: 指定所选择的文件。

parent: 指定该对话框的属主窗口。

title: 指定对话框的标题。

multiple: 指定是否允许多选。

defaulttextension：指定默认扩展名。当用户没有输入扩展名时，系统会默认添加该选项指定的扩展名。

filetypes：指定在该文件对话框中能查看的文件类型。该选项值是一个序列，可指定多个文件类型。可以通过"\*"指定浏览所有文件。

菜单 Menu

- 1、使用Menu类创建一个菜单
  - 2、add\_command添加菜单项，如果该菜单是顶层菜单，则添加的菜单项依次向右添加。 如果该菜单时顶层菜单的一个菜单项，则它添加的是下拉 菜单的菜单项。
- 属性：
- label：指定菜单的名称
  - command:被点击时调用的方法
  - acceleator：快捷键
  - underline：是否拥有下划线
- 3、窗口的menu属性：指定顶级菜单项

```
from tkinter import *
root = Tk() #定义一个顶级大窗口
menubar = Menu(root) #在大窗口下定义一个菜单实例
for each in ["文件", "视图", "编辑", "关于"]: #给菜单实例增加菜单项
 menubar.add_command(label=each)
root['menu']=menubar #菜单实例应用到大窗口中
root.mainloop()
```



```
from tkinter import *
#定义一个顶级大窗口
root = Tk()
#在大窗口下定义一个顶级菜单实例
menubar = Menu(root)

#在顶级菜单实例下创建子菜单实例
fmenu = Menu(menubar)
for each in ['新建', '打开', '保存', '另存为']:
 fmenu.add_command(label=each)

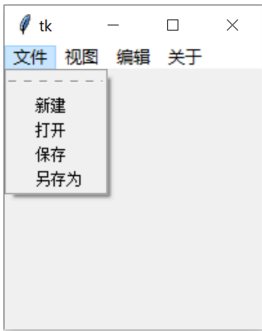
vmenu = Menu(menubar)
#为每个子菜单实例添加菜单项
for each in ['复制', '粘贴', '剪切']:
 vmenu.add_command(label=each)

emenu = Menu(menubar)
for each in ['默认视图', '新式视图']:
 emenu.add_command(label=each)

amenu = Menu(menubar)
for each in ['版权信息', '联系我们']:
 amenu.add_command(label=each)

#为顶级菜单实例添加菜单，并级联相应的子菜单实例
menubar.add_cascade(label='文件', menu=fmenu)
menubar.add_cascade(label='视图', menu=vmenu)
menubar.add_cascade(label='编辑', menu=emenu)
menubar.add_cascade(label='关于', menu=amenu)

#顶级菜单实例应用到大窗口中
root['menu']=menubar
root.mainloop()
```



```
from tkinter import *

root = Tk()

def callback():
 print("~被调用了~")

创建一个顶级菜单
menubar = Menu(root)

创建一个下拉菜单“文件”，然后将它添加到顶级菜单中
filemenu = Menu(menubar, tearoff=False)
filemenu.add_command(label="打开", command=callback)
filemenu.add_command(label="保存", command=callback)
filemenu.add_separator()
filemenu.add_command(label="退出", command=root.quit)
menubar.add_cascade(label="文件", menu=filemenu)

创建另一个下拉菜单“编辑”，然后将它添加到顶级菜单中
editmenu = Menu(menubar, tearoff=False)
editmenu.add_command(label="剪切", command=callback)
editmenu.add_command(label="拷贝", command=callback)
editmenu.add_command(label="粘贴", command=callback)
menubar.add_cascade(label="编辑", menu=editmenu)

显示菜单
root.config(menu=menubar)

mainloop()
```

下拉菜单中的每一项与鼠标事件的绑定

Tearoff

设置为true，表明这个菜单是可以独立

设置为False，表明这个菜单不可以独立

• 点菜小程序

1. 需求

- 左侧 Listbox 显示菜品 + 价格
- 选中后点 “点菜” 加入右侧 Listbox
- 自动计算总价并显示

2. 数据定义:

```
DictMenu={"香菇青菜":5,"红烧排骨":10,"清蒸带鱼":15,"宫爆鸡丁":10,"咖喱牛肉":20}
```

3. 核心逻辑

- 获取选中项: `theLB.curselection()`
- 插入客人菜单: `theGuestLB.insert("end",SelectionFood)`
- 累加价格: `price += DictMenu[SelectionFood]`

4. 完整代码

```
from tkinter import *
DictMenu={"香菇青菜":5,"红烧排骨":10,"清蒸带鱼":15,"宫爆鸡丁":10,"咖喱牛肉":20}
price=0
def AddFood():
 global price
 if theLB.curselection()!=(0):
 SelectionFood=theLB.get(theLB.curselection())
 theGuestLB.insert("end",SelectionFood)
 price += DictMenu[SelectionFood]
 thePrice.config(text="总价格%d"%price)
master = Tk()
theTitle=Label(master,text="餐厅点菜")
theTitle.grid(row=0,column=2,columnspan=2)
theLB= Listbox(master)
for item in DictMenu.items():
 theLB.insert("end",item)
theLB.grid(row=1,column=0,columnspan=2,rowspan=6)
theGuestLB = Listbox(master)
theGuestLB.grid(row=1,column=3,columnspan=2,rowspan=6)
button1=Button(master,text="点菜",command=AddFood)
button1.grid(row=8,column=2)
thePrice=Label(master,text="总价格")
thePrice.grid(row=9,column=2,columnspan=2)
master.mainloop()
```

一、什么时候必须用 `variable=var` ?

只有 3 种组件会用到，其他组件完全不需要：

表格

| 组件类型                                           | 场景     | 作用                    | 搭配变量类型                                        |
|------------------------------------------------|--------|-----------------------|-----------------------------------------------|
| <code>Radiobutton</code> 单选按钮                  | 分组互斥选择 | 实现「一组里只能选一个」，用变量记录选中项 | <code>IntVar</code> / <code>StringVar</code>  |
| <code>Checkbutton</code> 复选框                   | 记录勾选状态 | 记录「勾选 / 取消勾选」两种状态     | <code>IntVar</code> / <code>BooleanVar</code> |
| <code>Scale</code> / <code>Spinbox</code> 数值控件 | 记录数值   | 记录滑块 / 微调框的当前数值       | <code>IntVar</code> / <code>DoubleVar</code>  |

二、逐场景详细说明（对应你学过的内容）

1. `Radiobutton` 单选按钮（最常用）

✔ 必须用 `variable=var`，否则单选逻辑会失效

- 作用：把多个单选按钮绑定到同一个变量上，实现“互斥选择”
- 核心逻辑：每个按钮的 `value` 不同，选中哪个，变量就变成哪个按钮的 `value`
- 示例：  
👉 不写 `variable=var`，两个按钮会变成独立的，能同时选中，就不是单选了。

python

运行

```
var = tk.IntVar()
tk.Radiobutton(root, text="A选项", variable=var, value=1).pack()
tk.Radiobutton(root, text="B选项", variable=var, value=2).pack()
```

2. `Checkbutton` 复选框

✔ 必须用 `variable=var`，否则无法记录勾选状态

- 作用：把复选框的“勾选 / 取消”状态绑定到变量上
- 搭配参数：`onvalue`（选中时的值）和 `offvalue`（取消时的值）
- 示例：  
👉 不写 `variable=var`，你就没法通过代码判断用户有没有勾选。

python

运行

```
check_var = tk.IntVar()
tk.Checkbutton(root, text="记住我", variable=check_var, onvalue=1, offvalue=0).pack()
```

3. **Scale / Spinbox** 数值控件

✔ 可选，但强烈推荐用

- 作用：把滑块 / 微调框的当前数值绑定到变量上，方便代码实时获取和修改
- 示例：  
👉 不用也能通过 `.get()` 获取值，但用变量绑定会更方便，尤其是要监听数值变化的时候。

```
python
运行
scale_var = tk.IntVar()
tk.Scale(root, from_=0, to=100, variable=scale_var).pack()
```

三、什么时候不用 **variable=var** ？

你学过的大部分组件，**完全不需要**这个参数：

- `Label`、`Button`、`Entry`（注意：`Entry` 用的是 `textvariable`，不是 `variable`）
- `Canvas`、`Frame`、`Listbox`、`Menu` 等
- 这些组件要么直接通过 `.config()` 修改属性，要么用专门的参数绑定数据，和 `variable` 无关。

四、和 **textvariable** 的区别（你容易搞混的点）

很多同学会把这两个搞混，这里给你彻底分清：

表格

| 参数                        | 适用组件                                                                     | 绑定的内容      | 搭配变量                                          |
|---------------------------|--------------------------------------------------------------------------|------------|-----------------------------------------------|
| <code>variable</code>     | <code>Radiobutton</code> / <code>Checkbutton</code> / <code>Scale</code> | 组件的状态 / 数值 | <code>IntVar</code> / <code>BooleanVar</code> |
| <code>textvariable</code> | <code>Entry</code> / <code>Label</code> / <code>Button</code>            | 组件的文本内容    | <code>StringVar</code>                        |

五、一句话总结（背会就不会忘）

只有 3 种组件会用到 **variable=var**：单选、复选、数值滑块 / 微调框。它的核心作用，就是帮你记录这些组件的状态或数值，让你后续能方便地获取和处理用户的选择。